



UNIVERSIDADE
FEDERAL DO CARIRI

Transformações geométricas 2D

Professora Dra. Luana Batista da Cruz
luana.batista@ufca.edu.br

Roteiro

01 Introdução

02 Transformações básicas

03 Coordenadas homogêneas

04 Transformações inversas

05 Transformações 2D compostas

06 Outras transformações 2D

07 Transformações entre sistemas de coordenadas

08 Transformações 2D em OpenGL

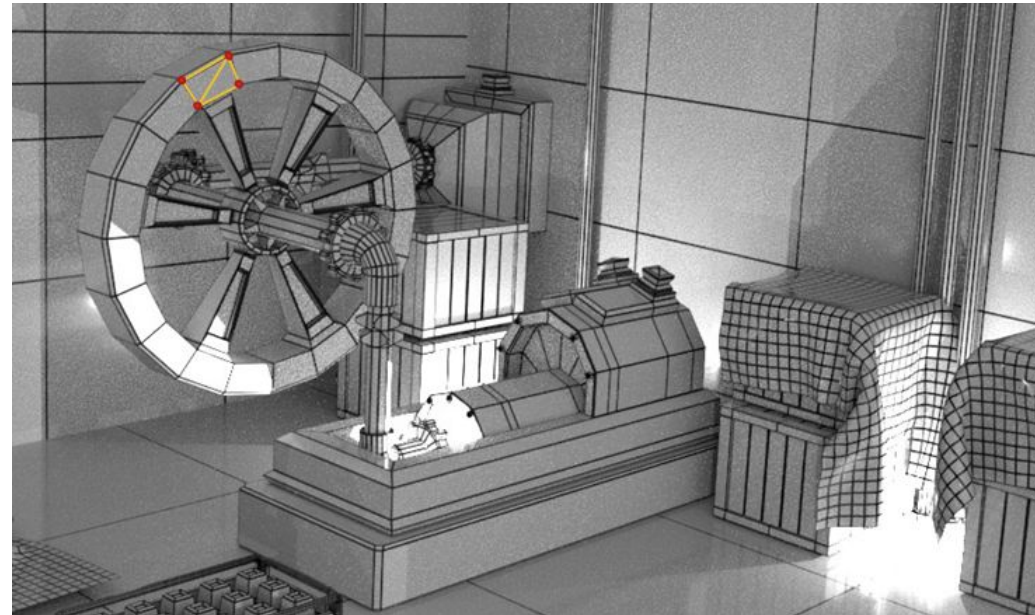
01

Introdução

Conceito
Matrizes

Introdução

- Transformações geométricas são operações aplicadas à descrição geométrica de um objeto para mudar sua
 - Posição (translação)
 - Orientação (rotação)
 - Tamanho (escala)



Introdução

- Além dessas transformações básicas, existem outras
 - Reflexão
 - Cisalhamento

Matrizes

- Uma **matriz A** de dimensões $m \times n$ é uma tabela composta por m linhas e n colunas, contendo $m \cdot n$ elementos numéricos

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & & \dots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

Matrizes

- Operações em matrizes
 - **Adição de matrizes**
 - As matrizes envolvidas devem ser da **mesma ordem** (mesmo número de linhas e colunas). E o resultado dessa soma será também outra matriz com a mesma ordem

$$\begin{bmatrix} 1 & 3 & 1 \\ 1 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 5 \\ 7 & 5 & 0 \end{bmatrix} = \begin{bmatrix} 1+0 & 3+0 & 1+5 \\ 1+7 & 0+5 & 0+0 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 6 \\ 8 & 5 & 0 \end{bmatrix}$$

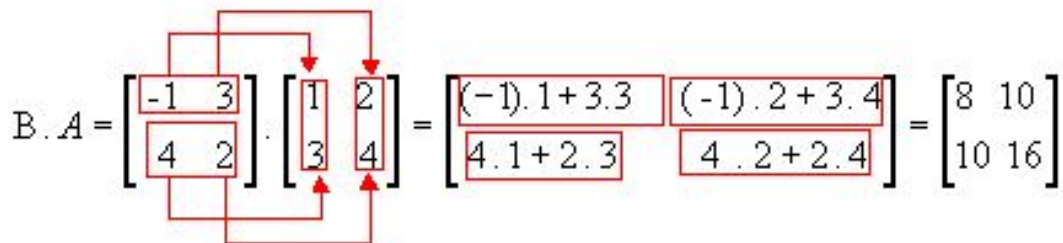
Matrizes

- Operações em matrizes
 - **Multiplicação da matriz por um escalar**
 - Multiplica o escalar por cada elemento da matriz

$$2 \cdot \begin{bmatrix} 1 & 8 & -3 \\ 4 & -2 & 5 \end{bmatrix} = \begin{bmatrix} 2 \cdot 1 & 2 \cdot 8 & 2 \cdot -3 \\ 2 \cdot 4 & 2 \cdot -2 & 2 \cdot 5 \end{bmatrix} = \begin{bmatrix} 2 & 16 & -6 \\ 8 & -4 & 10 \end{bmatrix}$$

Matrizes

- Operações em matrizes
 - **Multiplicação de matrizes**
 - Multiplica os elementos de cada linha da primeira matriz pelos elementos correspondentes das colunas da segunda e somando os resultados
 - Portanto, só é possível multiplicar duas matrizes quando o número de colunas da primeira é igual ao número de linhas da segunda
 - Resultado é uma matriz que tem o mesmo número de linhas da primeira e o mesmo número de colunas da segunda


$$B \cdot A = \begin{bmatrix} -1 & 3 \\ 4 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} (-1) \cdot 1 + 3 \cdot 3 & (-1) \cdot 2 + 3 \cdot 4 \\ 4 \cdot 1 + 2 \cdot 3 & 4 \cdot 2 + 2 \cdot 4 \end{bmatrix} = \begin{bmatrix} 8 & 10 \\ 10 & 16 \end{bmatrix}$$

Matrizes

- Operações em matrizes
 - **Multiplicação de matrizes**
 - Multiplicação de matrizes **não é comutativa!**

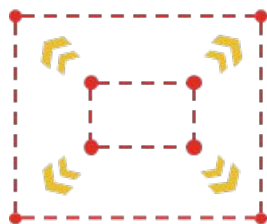
$$AB \neq BA$$

- Multiplicação de matrizes é **associativa**

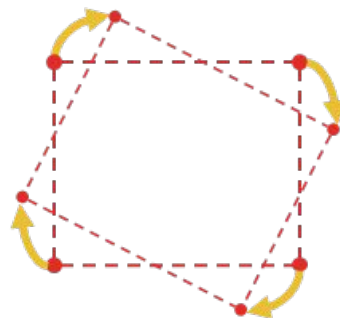
$$M_3 \cdot M_2 \cdot M_1 = (M_3 \cdot M_2) \cdot M_1 = M_3 \cdot (M_2 \cdot M_1)$$

Transformações

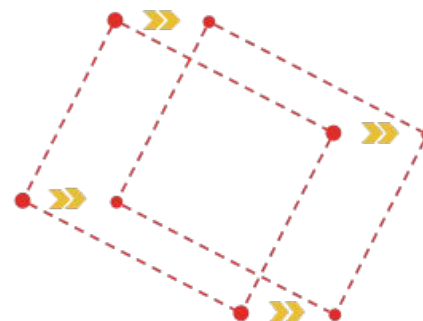
- As **transformações geométricas** são descritas e realizadas eficientemente através de **matrizes**



Escala



Rotação



Translação

02

Transformações básicas

Translação

Rotação

Escala

Translação

- A translação consiste em adicionar **offsets (deslocamento)** às coordenadas que definem um objeto

$$x' = x + t_x$$

$$y' = y + t_y$$

- Usando notação matricial, uma translação 2D pode ser descrita como

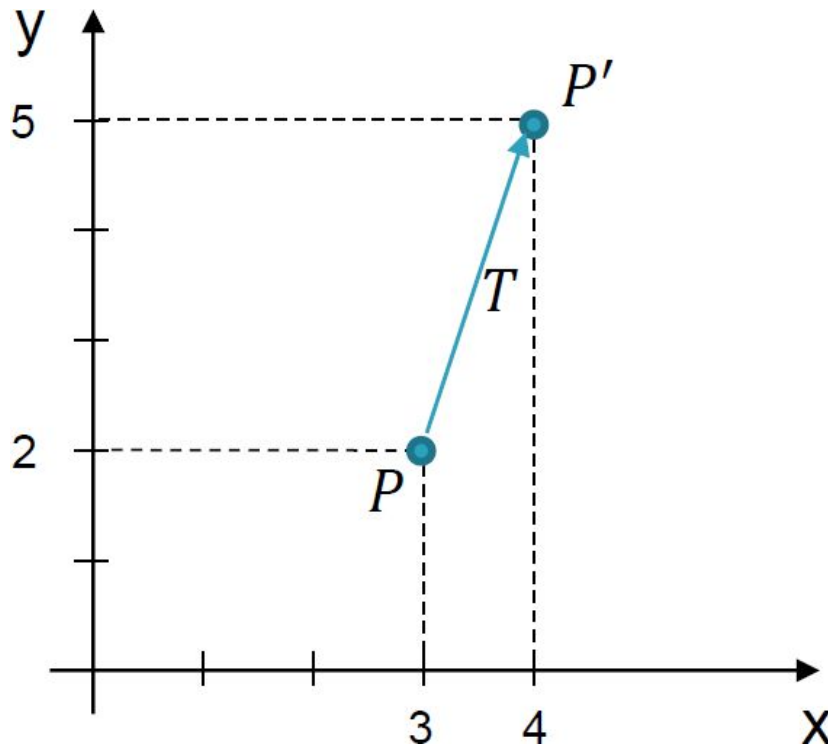
$$\mathbf{P}' = \mathbf{P} + \mathbf{T}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Translação

Exemplo

- Translação do ponto $P = (3, 2)$ com offsets $t_x = 1$ e $t_y = 3$

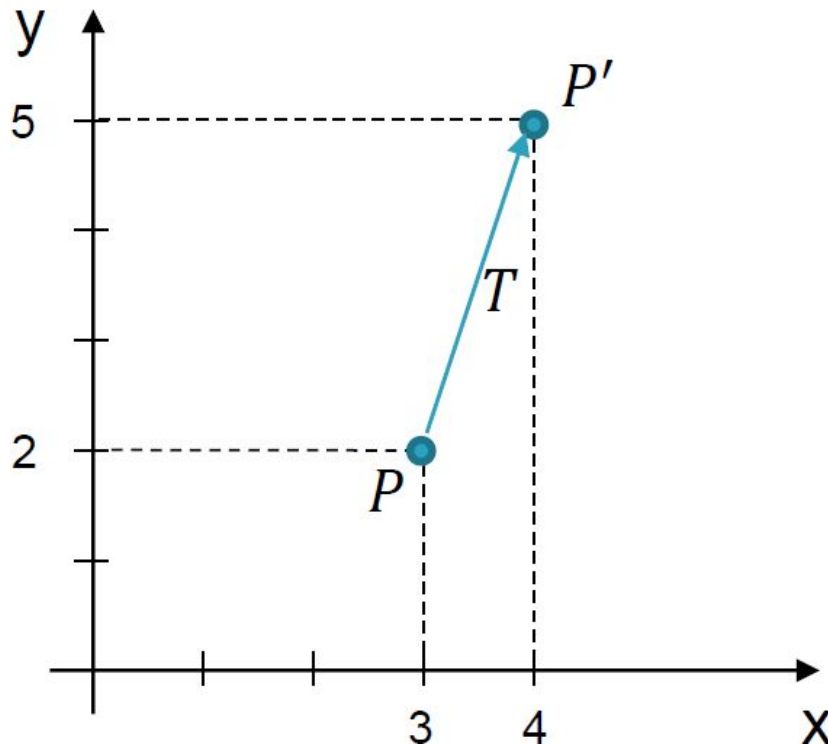


$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$
$$P' = (4, 5)$$

Translação

Exemplo

- Translação do ponto $P = (3, 2)$ com offsets $t_x = 1$ e $t_y = 3$



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

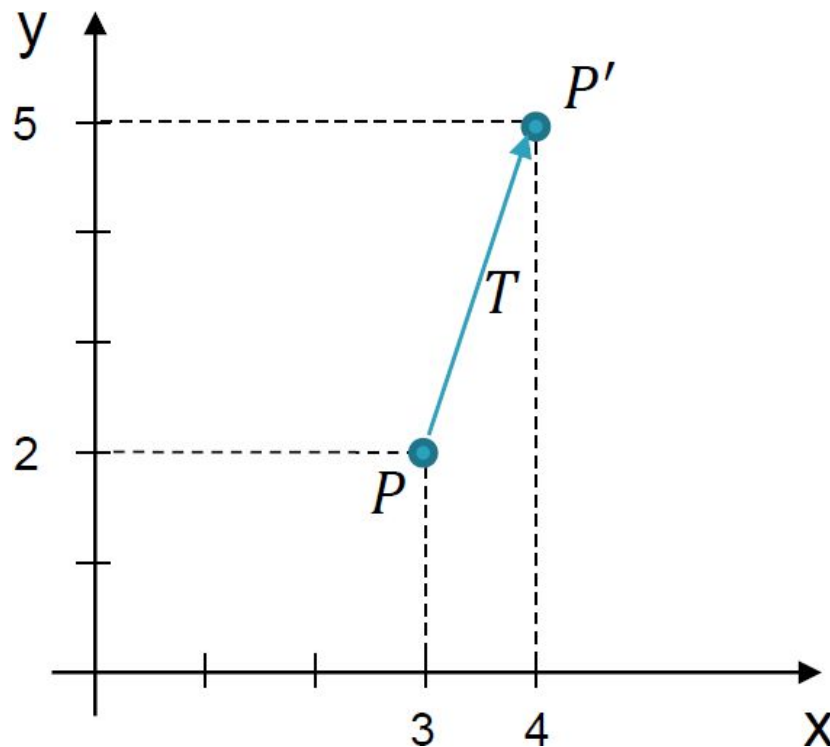
$$P' = (4, 5)$$

Se fosse uma linha?

Translação

Exemplo

- Translação do ponto $P = (3, 2)$ com offsets $t_x = 1$ e $t_y = 3$



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

$$P' = (4, 5)$$

Se fosse uma linha?

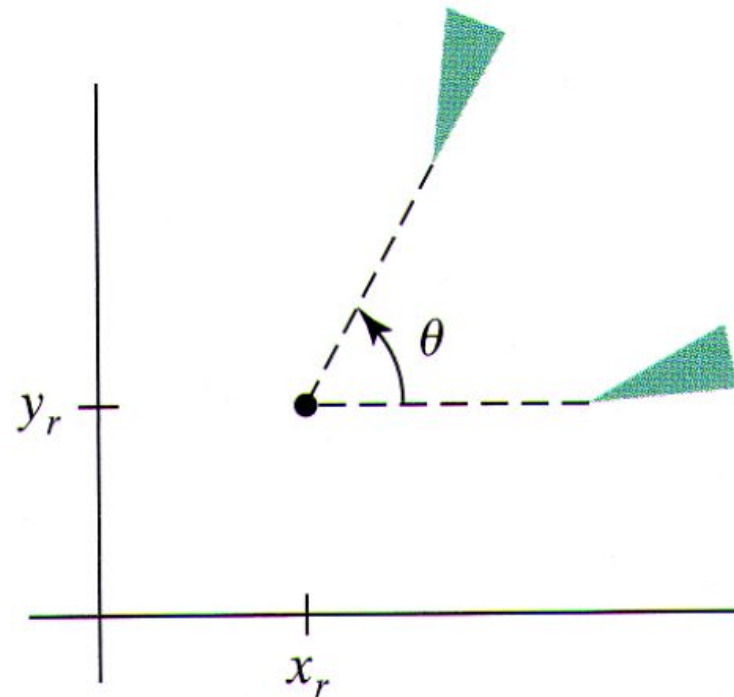
Aplicaria a mesma matriz de translação para todos os pontos

Rotação

- Define-se uma transformação de rotação por meio de um **eixo de rotação** e um **ângulo de rotação**
- Em 2D a rotação se dá em um caminho circular no plano, rotacionando o objeto considerando-se um eixo perpendicular ao plano xy

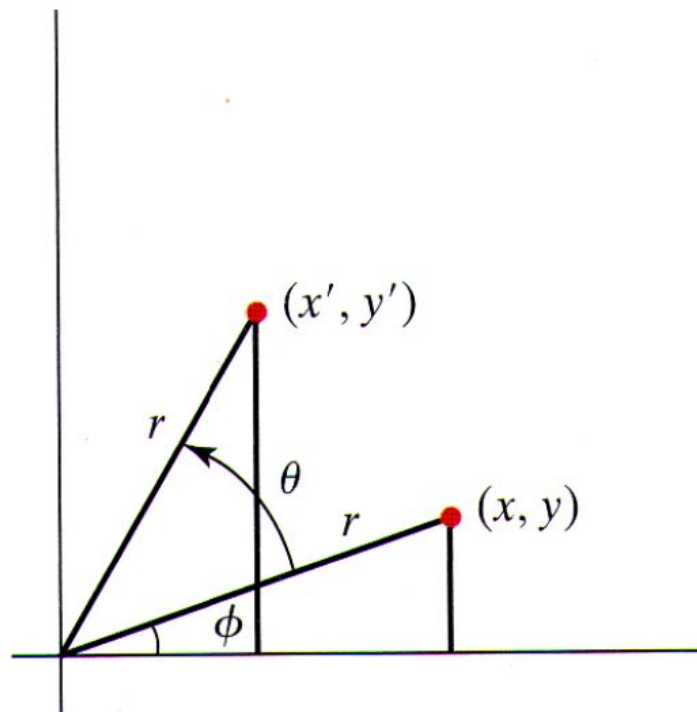
Rotação

- Parâmetros de rotação 2D são o ângulo de rotação (θ) e o ponto (x_r, y_r) de rotação, que é a intersecção do eixo de rotação com o plano xy
 - Se $\theta > 0$, a rotação é anti-horária
 - Se $\theta < 0$, a rotação é horária



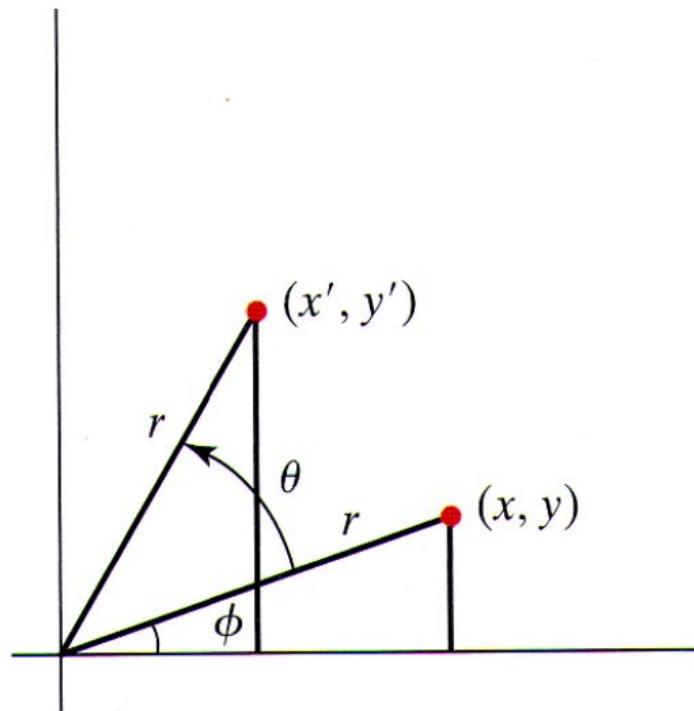
Rotação - Dedução

- Para simplificar considera-se que o ponto de rotação está na origem do sistema de coordenadas
 - O raio r é constante, ϕ é o ângulo original de $P = (x, y)$ e θ é o ângulo de rotação



Rotação - Dedução

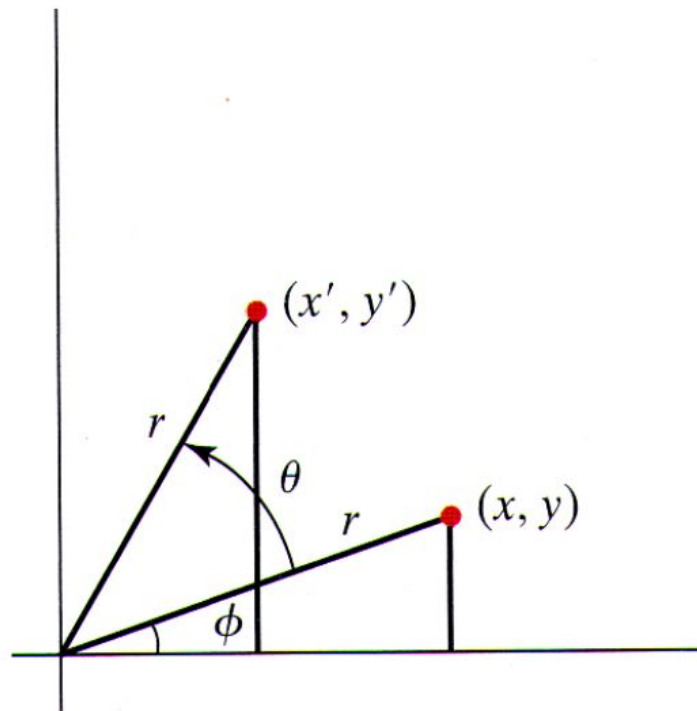
- Para simplificar considera-se que o ponto de rotação está na origem do sistema de coordenadas
 - O raio r é constante, ϕ é o ângulo original de $P = (x, y)$ e θ é o ângulo de rotação



O objetivo é encontrar x', y' por meio da rotação θ

Rotação - Dedução

- Para simplificar considera-se que o ponto de rotação está na origem do sistema de coordenadas
 - O raio r é constante, Φ é o ângulo original de $P = (x, y)$ e θ é o ângulo de rotação



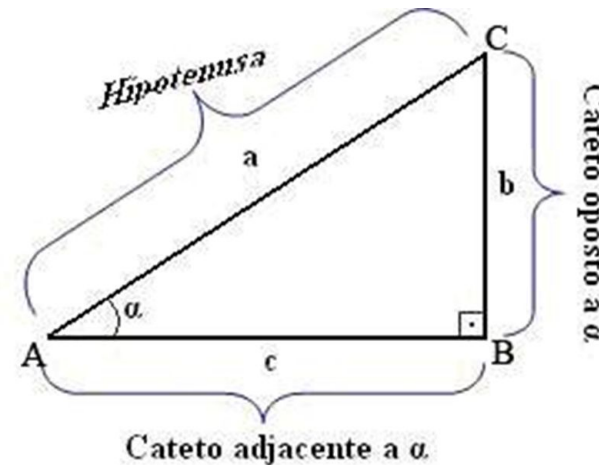
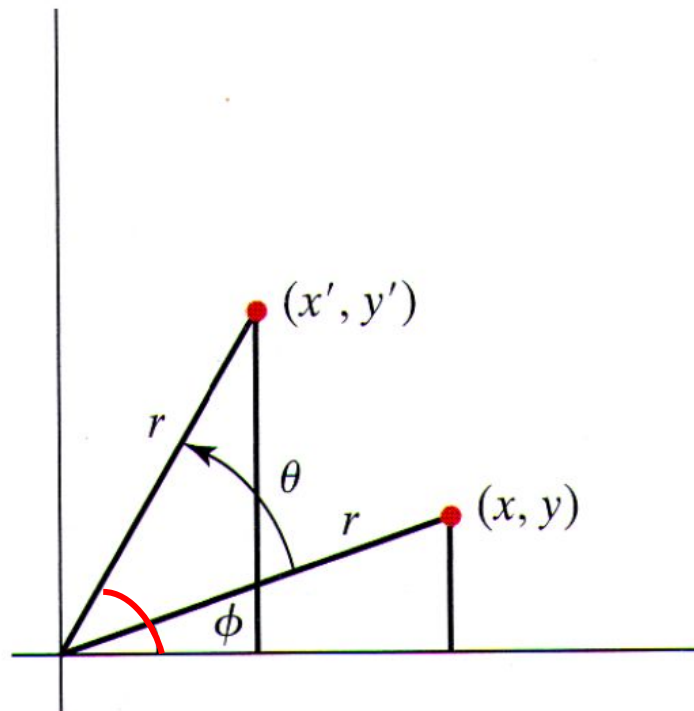
Deduziremos a **equação de rotação** usando os triângulos retângulos, seno, cosseno, Φ e θ

Rotação - Dedução

- Para simplificar considera-se que o ponto de rotação está na origem do sistema de coordenadas
 - O raio r é constante, Φ é o ângulo original de $P = (x, y)$ e θ é o ângulo de rotação

Primeiro analisaremos o primeiro triângulo retângulo

Cosseno e seno de um ângulo ($\Phi + \theta$)



Rotação - Dedução

- Sabendo que

$$\cos(\phi + \theta) = \frac{\text{cateto adjacente}}{\text{hipotenusa}} = \frac{x'}{r} \Rightarrow x' = r \cdot \cos(\phi + \theta)$$

$$\text{sen}(\phi + \theta) = \frac{\text{cateto oposto}}{\text{hipotenusa}} = \frac{y'}{r} \Rightarrow y' = r \cdot \text{sen}(\phi + \theta)$$

- Eliminar a somatória dos ângulos do seno e cosseno (identidade trigonométrica)

$$\cos(\alpha + \beta) = \cos(\alpha) \cdot \cos(\beta) - \text{sen}(\alpha) \text{sen}(\beta)$$

$$\text{sen}(\alpha + \beta) = \cos(\alpha) \cdot \text{sen}(\beta) + \text{sen}(\alpha) \cos(\beta)$$

- Então por substituição

$$x' = r \cdot \cos(\phi) \cdot \cos(\theta) - r \cdot \text{sen}(\phi) \cdot \text{sen}(\theta)$$

$$y' = r \cdot \cos(\phi) \cdot \text{sen}(\theta) + r \cdot \text{sen}(\phi) \cdot \cos(\theta)$$

Rotação - Dedução

- Sabendo que

$$\cos(\phi + \theta) = \frac{\text{cateto adjacente}}{\text{hipotenusa}} = \frac{x'}{r} \Rightarrow x' = r \cdot \cos(\phi + \theta)$$

$$\sin(\phi + \theta) = \frac{\text{cateto oposto}}{\text{hipotenusa}} = \frac{y'}{r} \Rightarrow y' = r \cdot \sin(\phi + \theta)$$

Já seriam as equações de rotação. No entanto, normalmente não temos o valor de r e ϕ

- Eliminar a somatória dos ângulos do seno e cosseno (identidade trigonométrica)

$$\cos(\alpha + \beta) = \cos(\alpha) \cdot \cos(\beta) - \sin(\alpha) \sin(\beta)$$

$$\sin(\alpha + \beta) = \cos(\alpha) \cdot \sin(\beta) + \sin(\alpha) \cos(\beta)$$

- Então por substituição

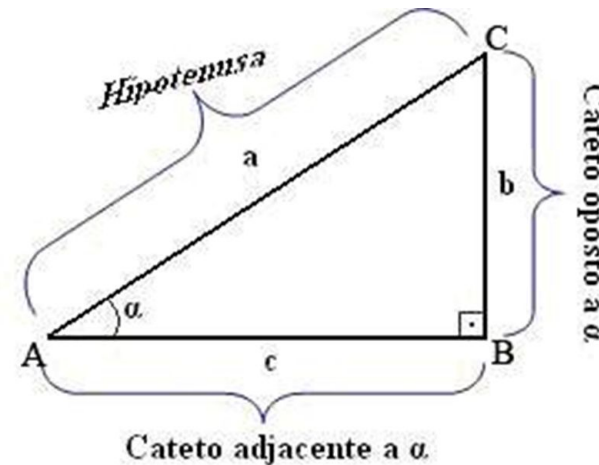
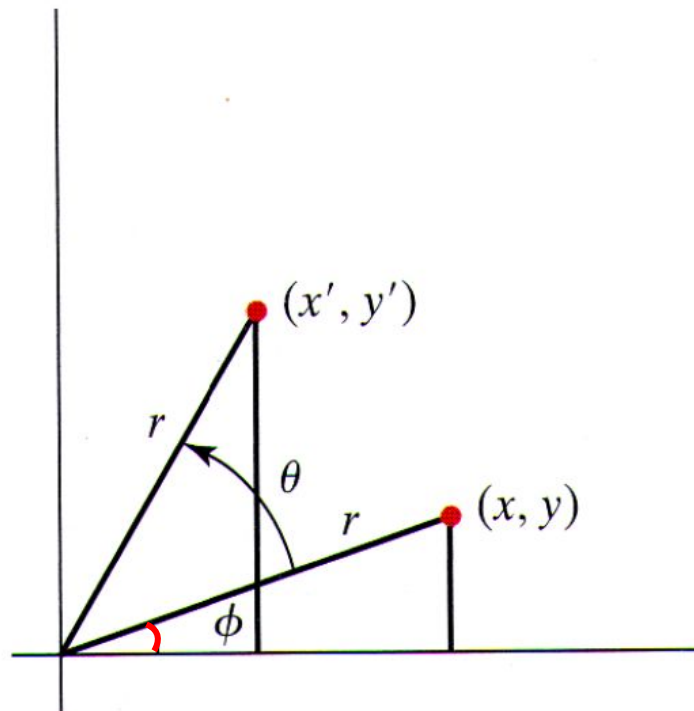
$$x' = r \cdot \cos(\phi) \cdot \cos(\theta) - r \cdot \sin(\phi) \cdot \sin(\theta)$$

$$y' = r \cdot \cos(\phi) \cdot \sin(\theta) + r \cdot \sin(\phi) \cdot \cos(\theta)$$

Rotação - Dedução

- Para simplificar considera-se que o ponto de rotação está na origem do sistema de coordenadas
 - O raio r é constante, Φ é o ângulo original de $P = (x, y)$ e θ é o ângulo de rotação

Podemos realizar o mesmo processo com o outro triângulo retângulo



Rotação - Dedução

- Lembrando que o ponto $P = (x, y)$ no sistema cartesiano também pode ser descrito por meio de coordenadas polares como

$$x = r \cdot \cos(\phi) \qquad y = r \cdot \sin(\phi)$$

- Então por substituição temos a **equação final da rotação**

$$\begin{aligned} x' &= r \cdot \cos(\phi) \cdot \cos(\theta) - r \cdot \sin(\phi) \cdot \sin(\theta) & \longrightarrow & & x' &= x \cdot \cos(\theta) - y \cdot \sin(\theta) \\ y' &= r \cdot \cos(\phi) \cdot \sin(\theta) + r \cdot \sin(\phi) \cdot \cos(\theta) & & & y' &= x \cdot \sin(\theta) + y \cdot \cos(\theta) \end{aligned}$$

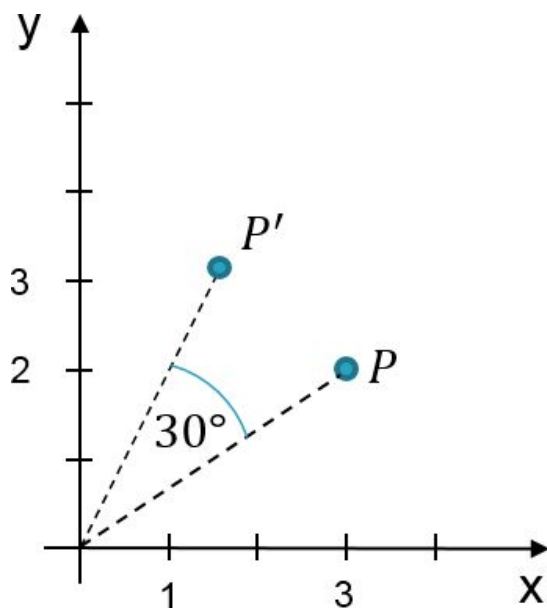
- Escrevendo na **forma matricial** temos

$$P' = R \cdot P$$
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Rotação

Exemplo

- Rotação do ponto $P = (3, 2)$ com ângulo de rotação $\theta = 30^\circ$ e ponto de rotação na origem do sistema de coordenadas



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(30^\circ) & -\sin(30^\circ) \\ \sin(30^\circ) & \cos(30^\circ) \end{bmatrix} \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 3 \cdot \cos(30^\circ) - 2 \cdot \sin(30^\circ) \\ 3 \cdot \sin(30^\circ) + 2 \cdot \cos(30^\circ) \end{bmatrix}$$

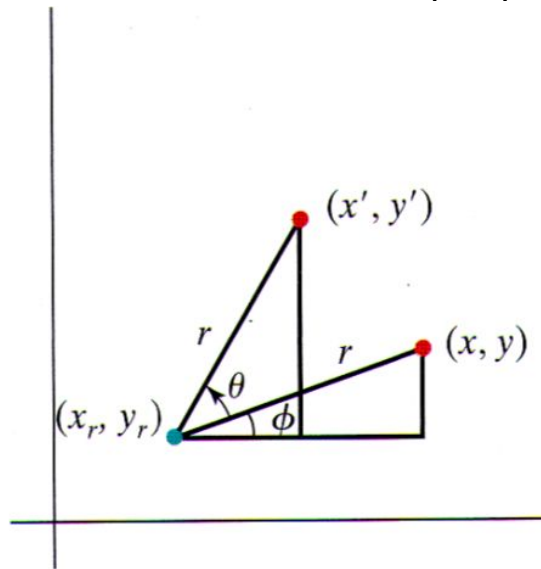
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 3 \cdot \sqrt{3}/2 - 2 \cdot 1/2 \\ 3 \cdot 1/2 + 2 \cdot \sqrt{3}/2 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1.598076 \\ 3.232051 \end{bmatrix}$$

- Dicas: $\cos(30^\circ) = \sqrt{3}/2$ e $\sin(30^\circ) = 1/2$

Rotação

- Rotação em torno de um ponto arbitrário (x_r, y_r)



- Poderíamos deduzir a rotação em torno de um ponto arbitrário da mesma forma que deduzimos a rotação em torno da origem do sistema de coordenadas
- No entanto, existe uma forma **melhor** e **mais fácil** de fazer isso que será apresentada mais adiante

Transformações: corpos rígidos e não rígidos

- A **translação**, **rotação** e **reflexão** são transformações de **corpo rígido** pois direcionam ou movem um objeto **sem deformá-lo**
 - Mantêm ângulos e distâncias entre as coordenadas do objeto, preservando sua forma original
- Por outro lado, transformações como a **escala** e **cisalhamento** são consideradas de **corpo não rígido**, pois podem alterar o tamanho ou a forma do objeto

Escala

- Para **alterar o tamanho** de um objeto aplica-se a transformação de **escala**
- Multiplica-se as coordenadas de um objeto por **fatores de escala**

$$x' = s_x \cdot x$$

$$y' = s_y \cdot y$$

- Na forma matricial

$$P' = S \cdot P$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

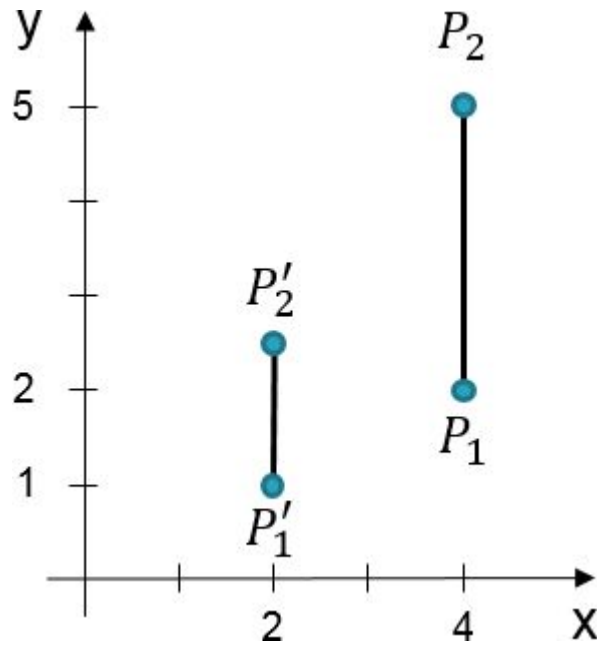
Escala

- Propriedades de s_x e s_y
 - s_x e s_y devem ser maiores que zero
 - $s_x > 1$ e $s_y > 1$, o objeto aumenta
 - $s_x < 1$ e $s_y < 1$, o objeto diminui
 - $s_x = s_y$, a escala é uniforme
 - $s_x \neq s_y$, a escala é diferencial

Escala

Exemplo

- Fazer a escala da reta definida por $P1 = (x1, y1) = (4, 2)$ e $P2 = (x2, y2) = (4, 5)$ com fatores de escala $s_x = s_y = 0.5$



$$\begin{bmatrix} x1' \\ y1' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x1 \\ y1 \end{bmatrix}$$

$$\begin{bmatrix} x1' \\ y1' \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 4 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} x1' \\ y1' \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \Rightarrow P1' = (2, 1)$$

$$\begin{bmatrix} x2' \\ y2' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x2 \\ y2 \end{bmatrix}$$

$$\begin{bmatrix} x2' \\ y2' \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

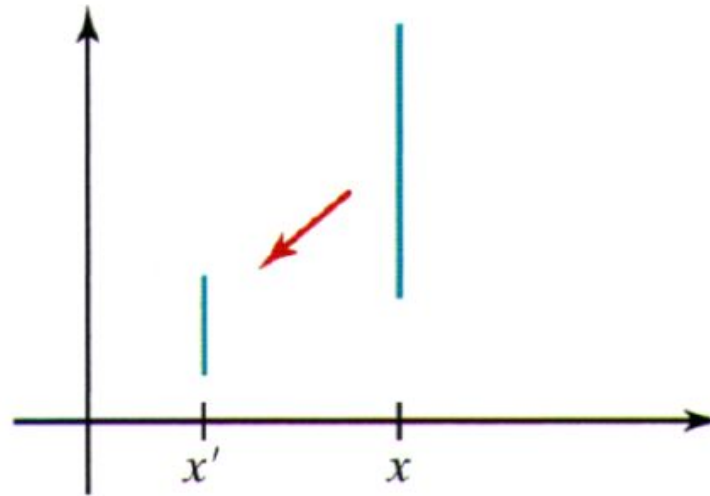
$$\begin{bmatrix} x2' \\ y2' \end{bmatrix} = \begin{bmatrix} 2 \\ 2.5 \end{bmatrix} \Rightarrow P2' = (2, 2.5)$$

Escala

Exemplo

- Pela formulação definida, o objeto é escalado e movido (efeito colateral)

Escala de uma linha
usando $s_x = s_y = 0.5$



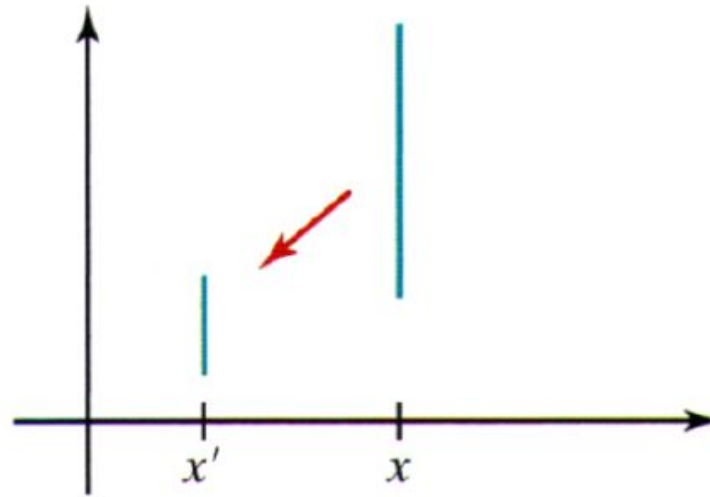
- É possível calcular uma forma matricial para fazer a escala com um ponto fixo
- No entanto, existe uma forma **melhor** e **mais fácil** de fazer isso que será apresentada mais adiante

Escala

Exemplo

- Pela formulação definida, o objeto é escalado e movido (efeito colateral)

Escala de uma linha
usando $s_x = s_y = 0.5$



Futuramente, iremos
aprender a fazer
rotação arbitrária e
escala com ponto fixo

- É possível calcular uma forma matricial para fazer a escala com um ponto fixo
- No entanto, existe uma forma **melhor** e **mais fácil** de fazer isso que será apresentada mais adiante

03

Coordenadas homogêneas

Translação 2D

Rotação 2D

Escala 2D

Coordenadas homogêneas

- As três transformações básicas podem ser expressas por

$$P' = M_1 \cdot P + M_2$$

- M_1 : matriz 2 x 2 com fatores multiplicativos (escala e rotação)
- M_2 : matriz coluna com fatores de adição (translação)
- Para se aplicar uma sequência de transformações, esse formato não ajuda
 - Eliminar a adição de matrizes permite escrever uma sequência de transformações com uma multiplicação de matrizes

Coordenadas homogêneas

- Isso pode ser feito expandindo-se para uma representação 3 x 3
- A **terceira coluna** é usada para os **termos da translação**
- **Coordenadas homogêneas**
 - Método para representar pontos e vetores em um espaço n-dimensional usando coordenadas adicionais
 - Uma forma de expansão do cálculo
 - Um ponto no espaço 2D representado em coordenadas homogêneas é descrito por 3 valores (x_h, y_h, h) , onde h é o parâmetro homogêneo ($h \neq 0$)

Coordenadas homogêneas

- A projeção do sistema homogêneo para o sistema cartesiano se dá pela seguinte relação

$$x = \frac{x_h}{h} \qquad y = \frac{y_h}{h}$$

- h pode ser qualquer valor diferente de zero, mas por conveniência, escolhemos $h = 1$ porque simplifica a conversão das coordenadas homogêneas
- Então as coordenadas homogêneas $(x_h, y_h, 1)$ ficam $(x, y, 1)$
- Usando coordenadas homogêneas, as transformações são convertidas em multiplicações de matrizes!

Coordenadas homogêneas

- A translação no espaço homogêneo é dada por

$$\begin{array}{l} x' = x + t_x \\ y' = y + t_y \end{array} \longrightarrow \begin{array}{l} x'_h = 1 \cdot x_h + 0 \cdot y_h + t_x \cdot h \\ y'_h = 0 \cdot x_h + 1 \cdot y_h + t_y \cdot h \\ h = 0 \cdot x_h + 0 \cdot y_h + 1 \cdot h \end{array}$$

Coordenadas homogêneas

- Por conveniência, com $h = 1$, definimos a **translação 2D** usando coordenadas homogêneas na forma matricial por

$$P' = T(t_x, t_y) \cdot P$$
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Coordenadas homogêneas

- Uma **rotação 2D** pode ser definida usando coordenadas homogêneas na forma matricial

$$P' = R(\theta) \cdot P$$
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Coordenadas homogêneas

- Uma **escala 2D** pode ser definida usando coordenadas homogêneas na forma matricial

$$P' = S(s_x, s_y) \cdot P$$
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

04

Transformações inversas

Translação inversa

Rotação inversa

Escala inversa

Translação inversa

- Para qualquer transformação básica que fizemos, temos a sua operação inversa, que é desfazer a operação que foi feita
- Para a **translação inversa 2D**, inverte-se o sinal dos offsets das translações

$$T^{-1} = \begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Rotação inversa

- Uma **rotação inversa 2D** é obtida trocando o ângulo de rotação por seu negativo

$$P' = R(\theta) \cdot P$$
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Isso rotaciona no sentido horário
- $R^{-1} = R^T$

Escala inversa

- O **inverso da escala 2D** é obtido trocando os fatores de escala por seus inversos

$$S^{-1} = \begin{bmatrix} 1/s_x & 0 & 0 \\ 0 & 1/s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Transformações 2D compostas

Introdução

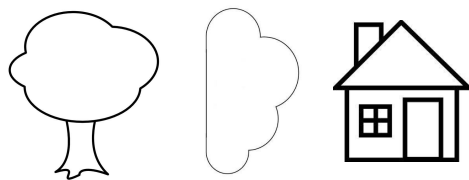
Rotação 2D com ponto de rotação arbitrário

Escala 2D com ponto fixo

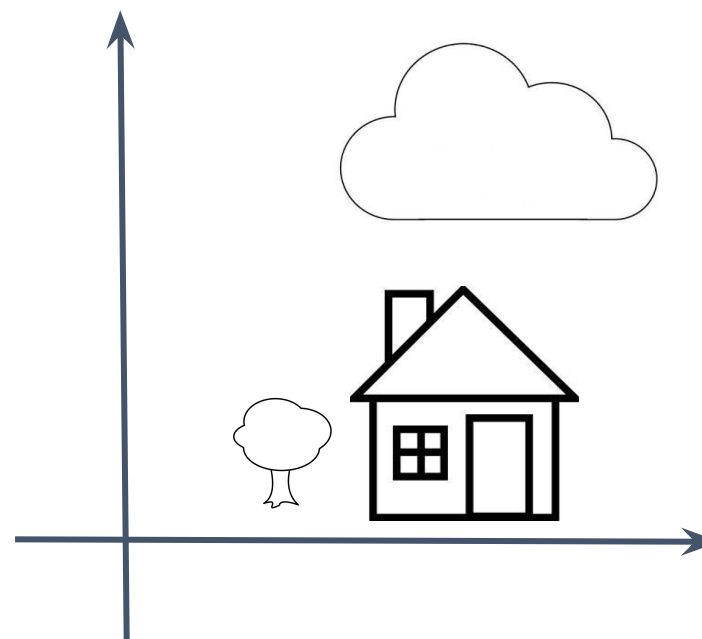
Propriedade da concatenação de matrizes

Introdução

- Muitas vezes, objetos são modificados e posicionados por várias transformações geométricas para compor uma cena



Imagine o cenário em que todos esses objetos começaram na origem



Introdução

- Muitas vezes, objetos são modificados e posicionados por várias transformações geométricas para compor uma cena
 - <https://www.youtube.com/watch?v=bnYSkJSs1h0>



Introdução

- Usando representações matriciais homogêneas, uma **sequência de transformações** pode ser representada como uma única matriz obtida a partir de multiplicações de matrizes de transformação

$$P' = M_2 \cdot M_1 \cdot P$$

$$P' = (M_2 \cdot M_1) \cdot P$$

$$P' = M \cdot P$$

- A transformação é dada por M ao invés de M_1 e M_2

Introdução

- Usando representações matriciais homogêneas, uma **sequência de transformações** pode ser representada como uma única matriz obtida a partir de multiplicações de matrizes de transformação

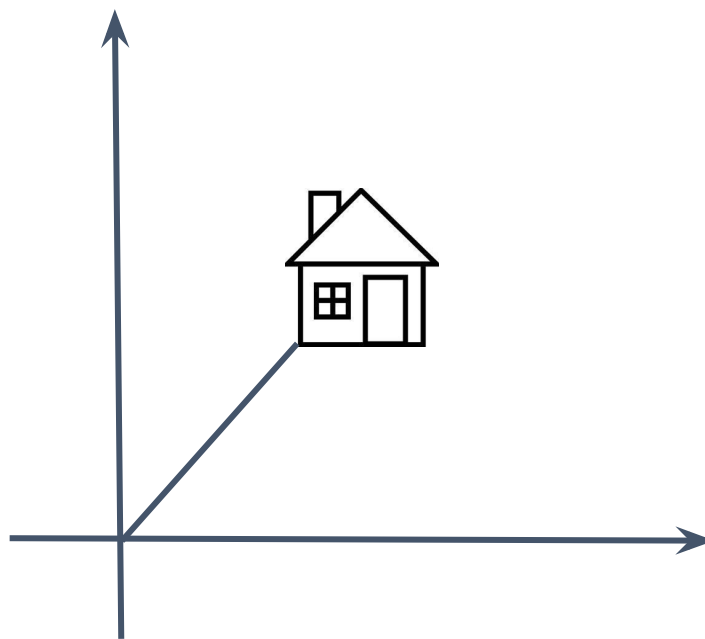
$$\begin{aligned}P' &= M_2 \cdot M_1 \cdot P \\P' &= (M_2 \cdot M_1) \cdot P \\P' &= M \cdot P\end{aligned}$$

Multiplicação de matrizes linha por coluna **não é comutativa**, ou seja, se multiplicar uma matriz A por B é diferente de multiplicar B por A

- A transformação é dada por M ao invés de M_1 e M_2

Rotação 2D em torno da origem

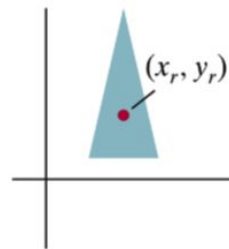
- A rotação tem um efeito colateral quando aplicada em objetos não centrados na origem



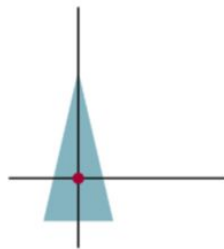
Rotaciona em torno da origem

Rotação 2D com ponto de rotação arbitrário

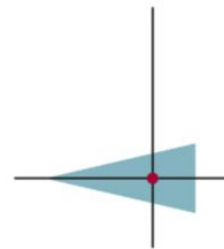
- Rotação com **ponto de rotação/pivô arbitrário** (x_r, y_r) é feita combinando-se múltiplas transformações



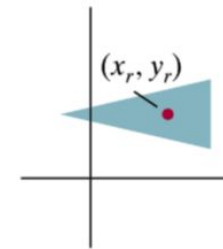
(a)
Original Position
of Object and
Pivot Point



(b)
Translation of
Object so that
Pivot Point
 (x_r, y_r) is at
Origin



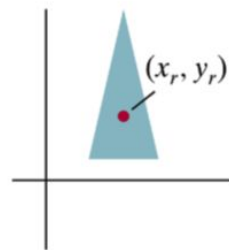
(c)
Rotation
about
Origin



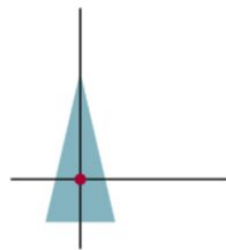
(d)
Translation of
Object so that
the Pivot Point
is Returned
to Position
 (x_r, y_r)

Rotação 2D com ponto de rotação arbitrário

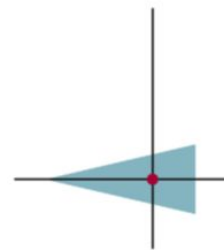
- Rotação com **ponto de rotação/pivô arbitrário** (x_r, y_r) é feita combinando-se múltiplas transformações



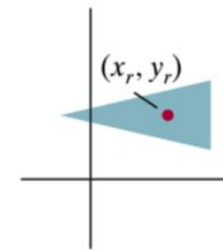
(a)
Original Position
of Object and
Pivot Point



(b)
Translation of
Object so that
Pivot Point
 (x_r, y_r) is at
Origin



(c)
Rotation
about
Origin



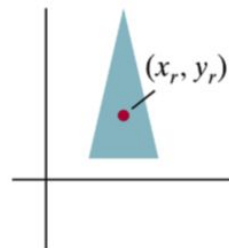
(d)
Translation of
Object so that
the Pivot Point
is Returned
to Position
 (x_r, y_r)

Como fazer esse processo pensando em reduzir cálculos?

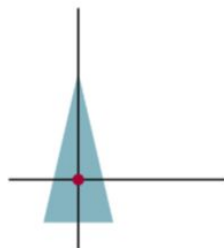
Rotação 2D com ponto de rotação arbitrário

- Rotação com **ponto de rotação/pivô arbitrário** (x_r, y_r) é feita combinando-se múltiplas transformações
 - 1) Mova o ponto de rotação para a origem (translação de ida)
 - 2) Execute a rotação
 - 3) Mova o ponto de rotação para a posição inicial (translação de volta)

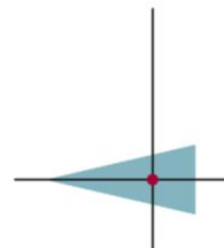
Ponto de rotação:
um dos vértices do
objeto ou o
centróide do objeto



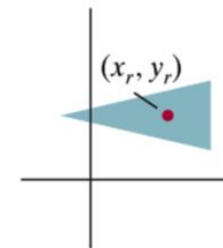
(a)
Original Position
of Object and
Pivot Point



(b)
Translation of
Object so that
Pivot Point
 (x_r, y_r) is at
Origin



(c)
Rotation
about
Origin



(d)
Translation of
Object so that
the Pivot Point
is Returned
to Position
 (x_r, y_r)

Rotação 2D com ponto de rotação arbitrário

- Rotação com **ponto de rotação/pivô arbitrário** (x_r, y_r) é feita combinando-se múltiplas transformações
 - 1) Mova o ponto de rotação para a origem (translação de ida)
 - 2) Execute a rotação
 - 3) Mova o ponto de rotação para a posição inicial (translação de volta)
- Em notação matricial

$$R(x_r, y_r, \theta) = T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r) =$$
$$\begin{bmatrix} 1 & 0 & x_r \\ 0 & 1 & y_r \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_r \\ 0 & 1 & -y_r \\ 0 & 0 & 1 \end{bmatrix}$$

As transformações são especificadas na ordem inversa em que são aplicadas

Rotação 2D com ponto de rotação arbitrário

- Rotação com **ponto de rotação/pivô arbitrário** (x_r, y_r) é feita combinando-se múltiplas transformações
 - 1) Mova o ponto de rotação para a origem (translação de ida)
 - 2) Execute a rotação
 - 3) Mova o ponto de rotação para a posição inicial (translação de volta)
- Em notação matricial

$$R(x_r, y_r, \theta) = T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r) =$$
$$\begin{bmatrix} 1 & 0 & x_r \\ 0 & 1 & y_r \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_r \\ 0 & 1 & -y_r \\ 0 & 0 & 1 \end{bmatrix}$$

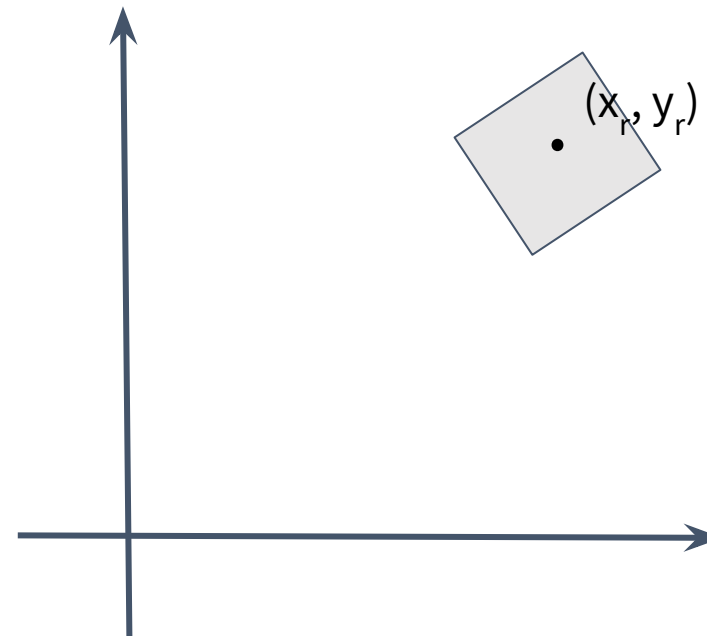
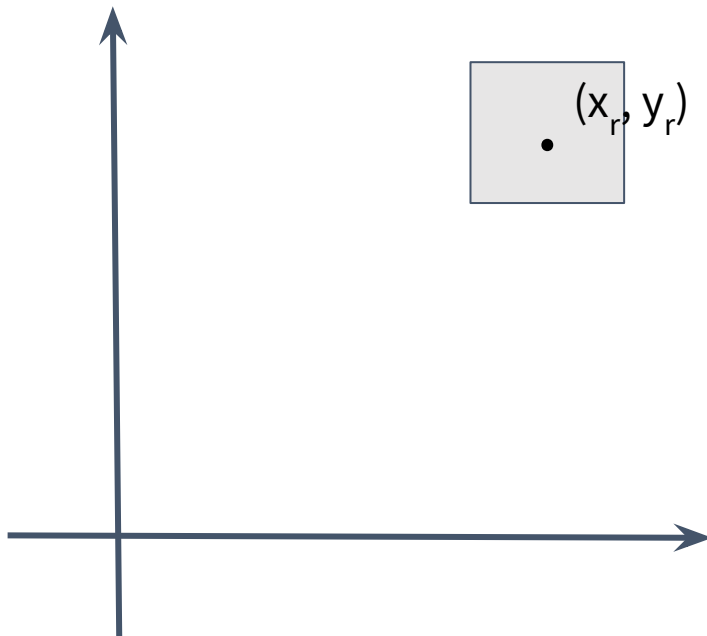
Esse processo é feito para um ponto (normalmente o central), gerando uma **matriz composta** que é usada para aplicar a todos os pontos

Rotação 2D com ponto de rotação arbitrário

- Mas não vai dar pra perceber que toda vez que rotacionar o objeto vai pro centro do sistema de coordenadas?

Rotação 2D com ponto de rotação arbitrário

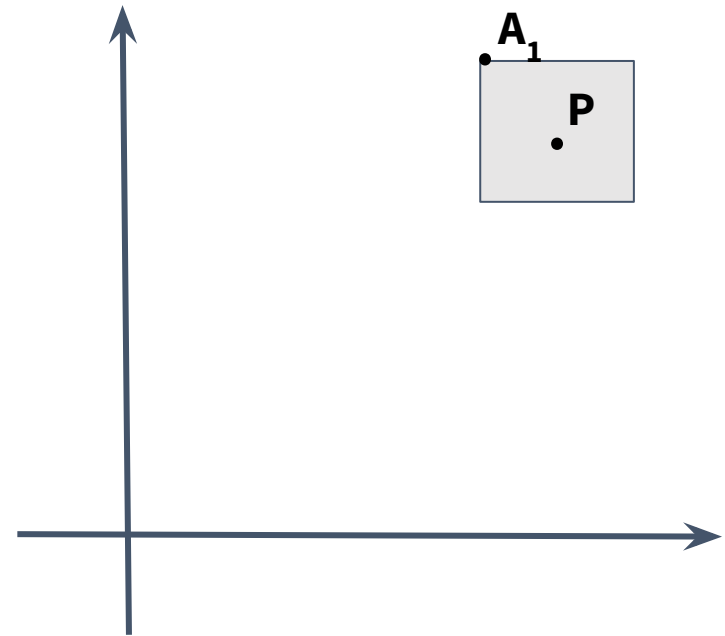
- Mas não vai dar pra perceber que toda vez que rotacionar o objeto vai pro centro do sistema de coordenadas?
 - Não, pois não é exibido na tela as etapas intermediárias (matriz composta)



Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

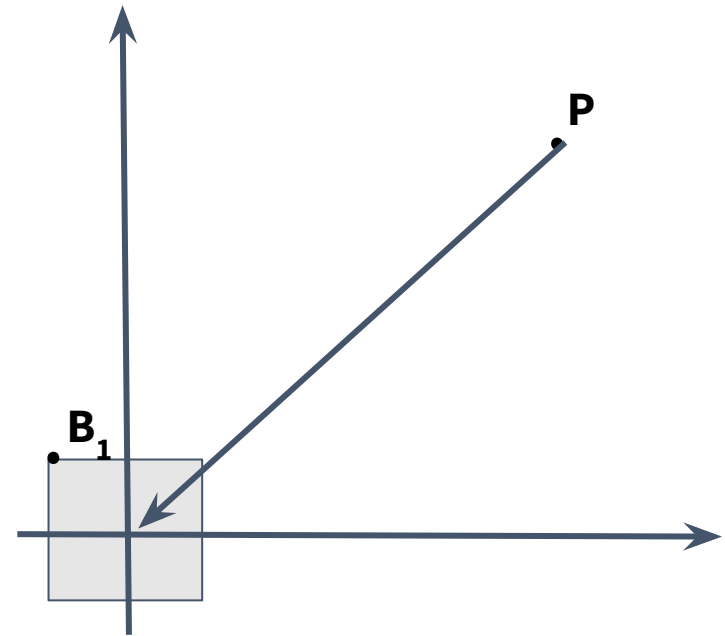
Translação para origem $T_{ida} \cdot A_1$



Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

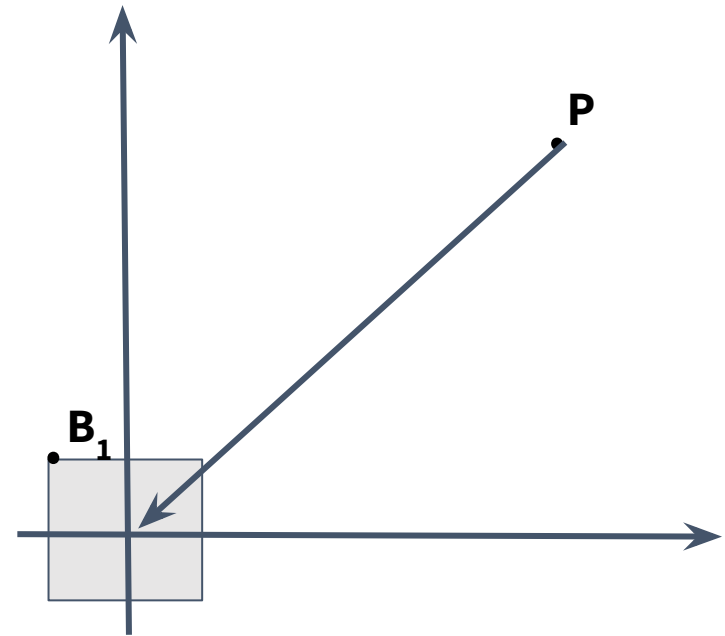


Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1$

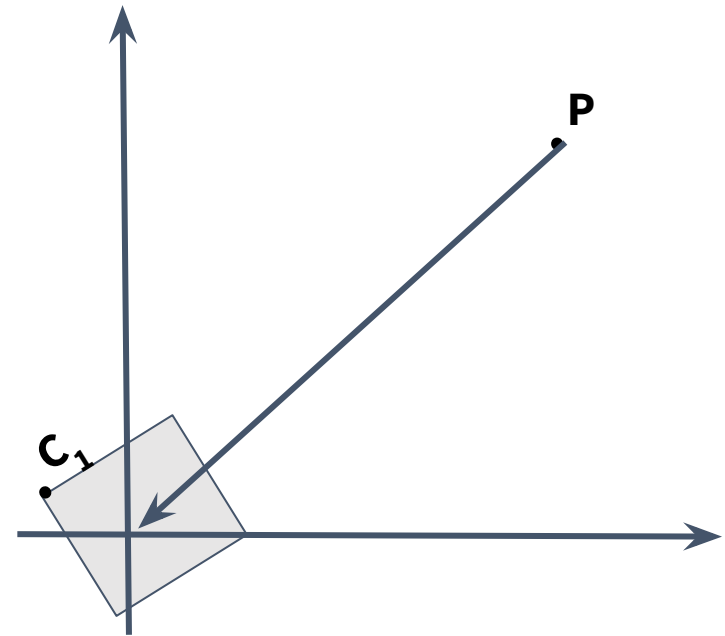


Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1 = C_1$



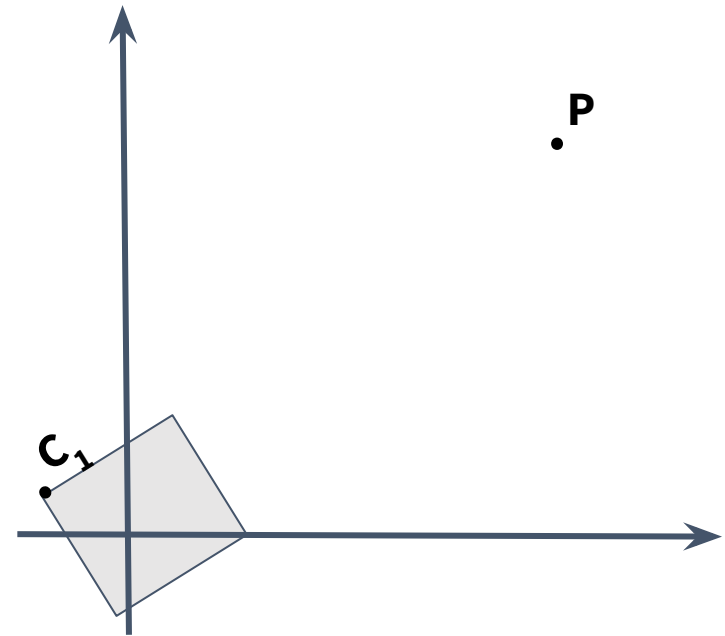
Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1$



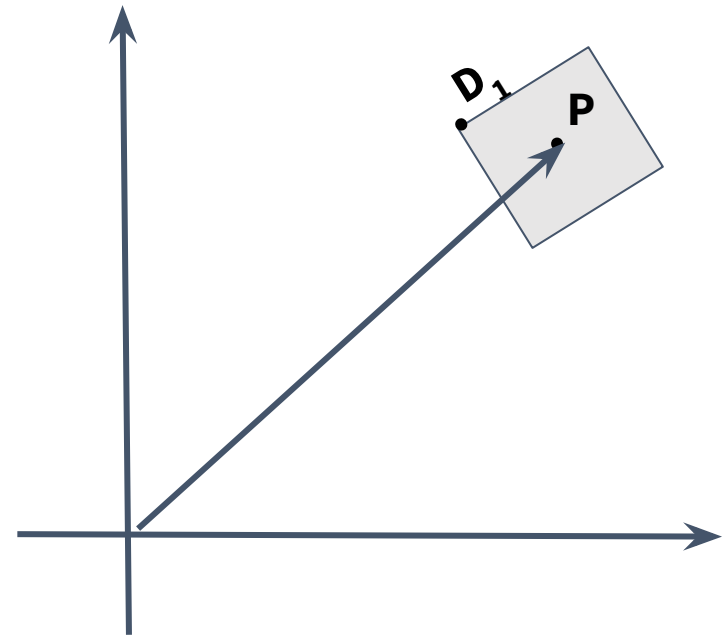
Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1 = D_1$



Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1 = D_1$

Vamos remover esses pontos intermediários fazendo uma série de substituições nessas equações para obter a matriz composta!

Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1 = D_1$

Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1 = D_1$

$R \cdot T_{ida} \cdot A_1 = C_1$

Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

$$R \cdot T_{ida} \cdot A_1 = C_1$$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1 = D_1$

Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

$$R \cdot T_{ida} \cdot A_1 = C_1$$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1 = D_1$

$$T_{volta} \cdot R \cdot T_{ida} \cdot A_1 = D_1$$

Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

$$R \cdot T_{ida} \cdot A_1 = C_1$$

Rotação $R \cdot B_1 = C_1$

$$T_{volta} \cdot R \cdot T_{ida} \cdot A_1 = D_1$$

Translação de volta $T_{volta} \cdot C_1 = D_1$

O que acontece se multiplicarmos uma matriz 3x3 por outra 3x3?

Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1 = D_1$

$$R \cdot T_{ida} \cdot A_1 = C_1$$

$$T_{volta} \cdot R \cdot T_{ida} \cdot A_1 = D_1$$

$$M_c = T_{volta} \cdot R \cdot T_{ida}$$

Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1 = D_1$

$$R \cdot T_{ida} \cdot A_1 = C_1$$

$$T_{volta} \cdot R \cdot T_{ida} \cdot A_1 = D_1$$

$$M_c = T_{volta} \cdot R \cdot T_{ida}$$

$3^\circ \quad 2^\circ \quad 1^\circ$

Rotação 2D com ponto de rotação arbitrário

- As matrizes são combinadas para formar a matriz composta
 - A saída de uma transformação é usada na próxima

Translação para origem $T_{ida} \cdot A_1 = B_1$

Rotação $R \cdot B_1 = C_1$

Translação de volta $T_{volta} \cdot C_1 = D_1$

$$R \cdot T_{ida} \cdot A_1 = C_1$$

$$T_{volta} \cdot R \cdot T_{ida} \cdot A_1 = D_1$$

$$M_c = T_{volta} \cdot R \cdot T_{ida}$$

3° 2° 1°

Dessa forma, podemos construir uma matriz composta para depois aplicar a todos os pontos

Propriedade da concatenação de matrizes

- Portanto, é possível compor transformações por causa das representações matriciais homogêneas
 - Todas as matrizes agora são normalizadas (3×3)
 - Pode fazer várias transformações multiplicando todas as matrizes porque satisfaz a propriedade da multiplicação de matrizes, resultando em uma única representação para aplicar a todos os pontos

Propriedade da concatenação de matrizes

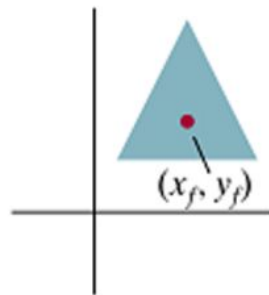
- Lembrando que multiplicação de matrizes é **associativa**

$$M_3 \cdot M_2 \cdot M_1 = (M_3 \cdot M_2) \cdot M_1 = M_3 \cdot (M_2 \cdot M_1)$$

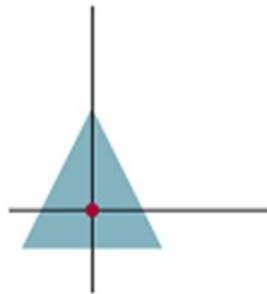
- Na pós-multiplicação, as transformações são especificadas na ordem inversa em que são aplicadas ($M_3 \rightarrow M_2 \rightarrow M_1$)
- O OpenGL também usa pós-multiplicação

Escala 2D com ponto fixo

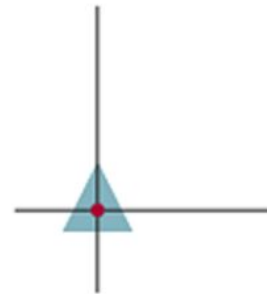
- Escala com **ponto fixo** (x_f, y_f) também é feita combinando-se múltiplas transformações
 - 1) Mova o ponto fixo para a origem (translação)
 - 2) Execute a escala
 - 3) Mova o ponto fixo para sua posição original (translação)



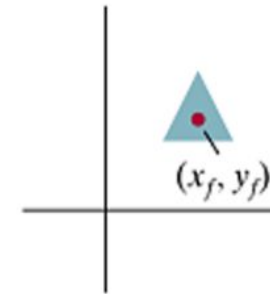
(a)
Original Position
of Object and
Fixed Point



(b)
Translate Object
so that Fixed Point
 (x_f, y_f) is at Origin



(c)
Scale Object
with Respect
to Origin



(d)
Translate Object
so that the Fixed
Point is Returned
to Position (x_f, y_f)

Escala 2D com ponto fixo

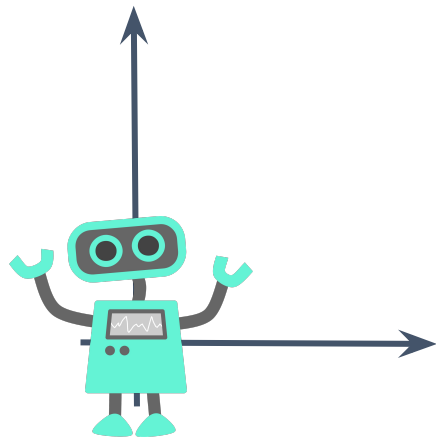
- Escala com **ponto fixo** (x_f, y_f) também é feita combinando-se múltiplas transformações
 - 1) Mova o ponto fixo para a origem (translação)
 - 2) Execute a escala
 - 3) Mova o ponto fixo para sua posição original (translação)
- Em notação matricial

$$S(x_f, y_f, s_x, s_y) = T(x_f, y_f) \cdot S(s_x, s_y) \cdot T(-x_f, -y_f)$$

$$\begin{bmatrix} 1 & 0 & x_f \\ 0 & 1 & y_f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_f \\ 0 & 1 & -y_f \\ 0 & 0 & 1 \end{bmatrix}$$

Após as etapas, crie a **matriz composta** e aplique a todos os pontos

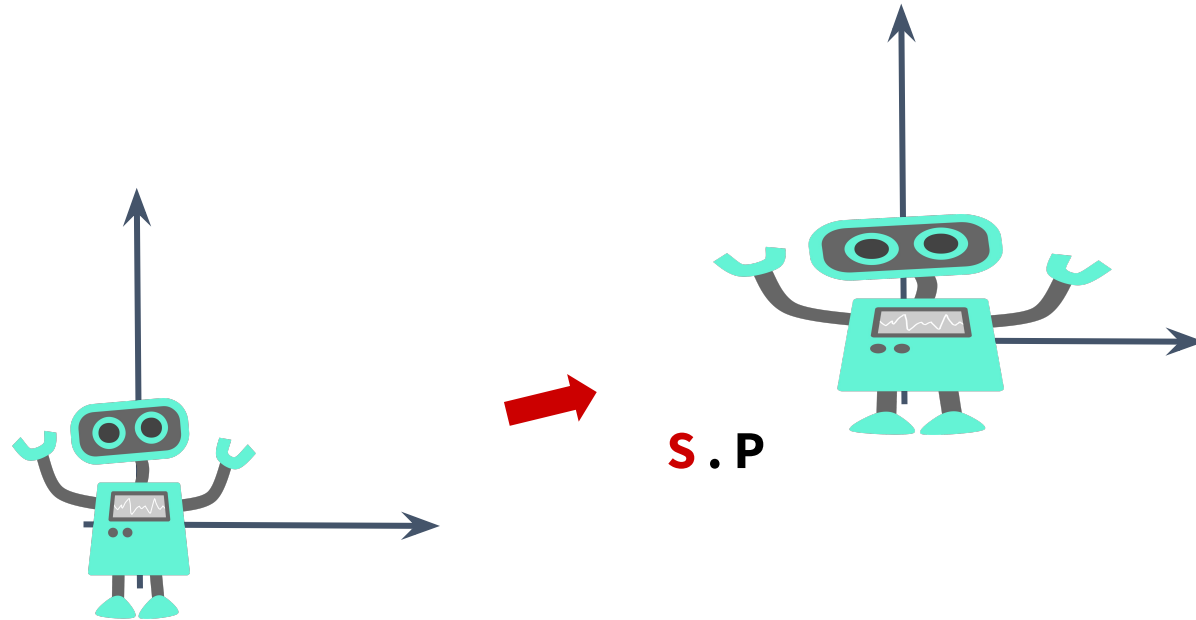
A ordem dos fatores altera o resultado



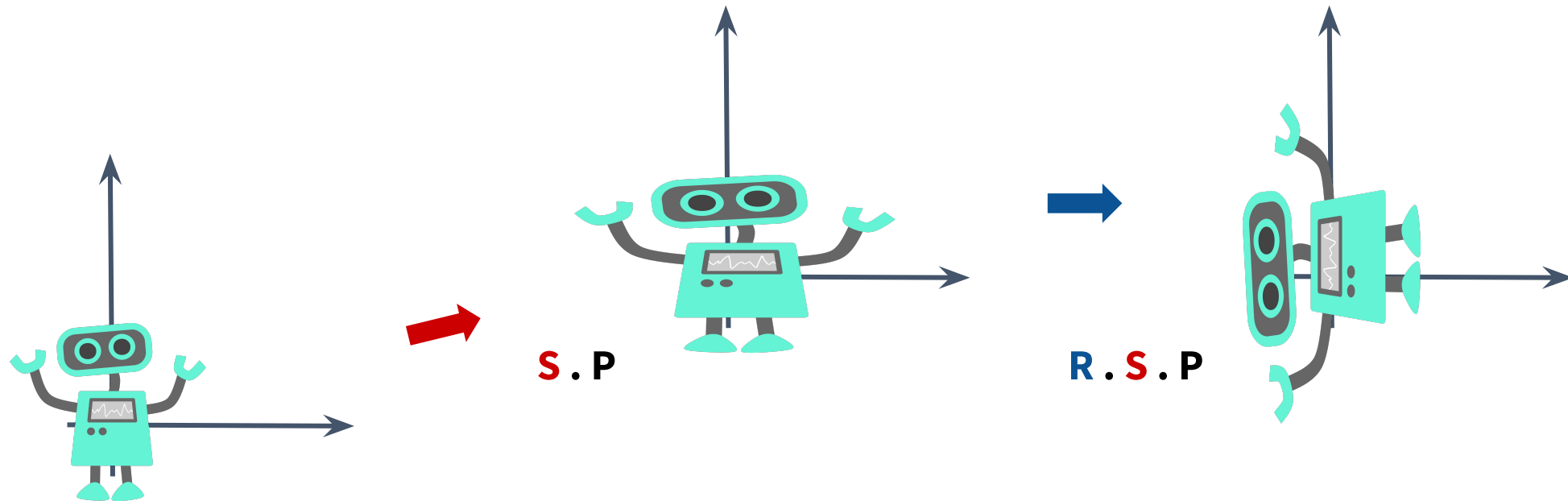
Duas transformações

- Escala não uniforme (apenas no eixo x)
- Rotação 90° (sentido anti-horário)

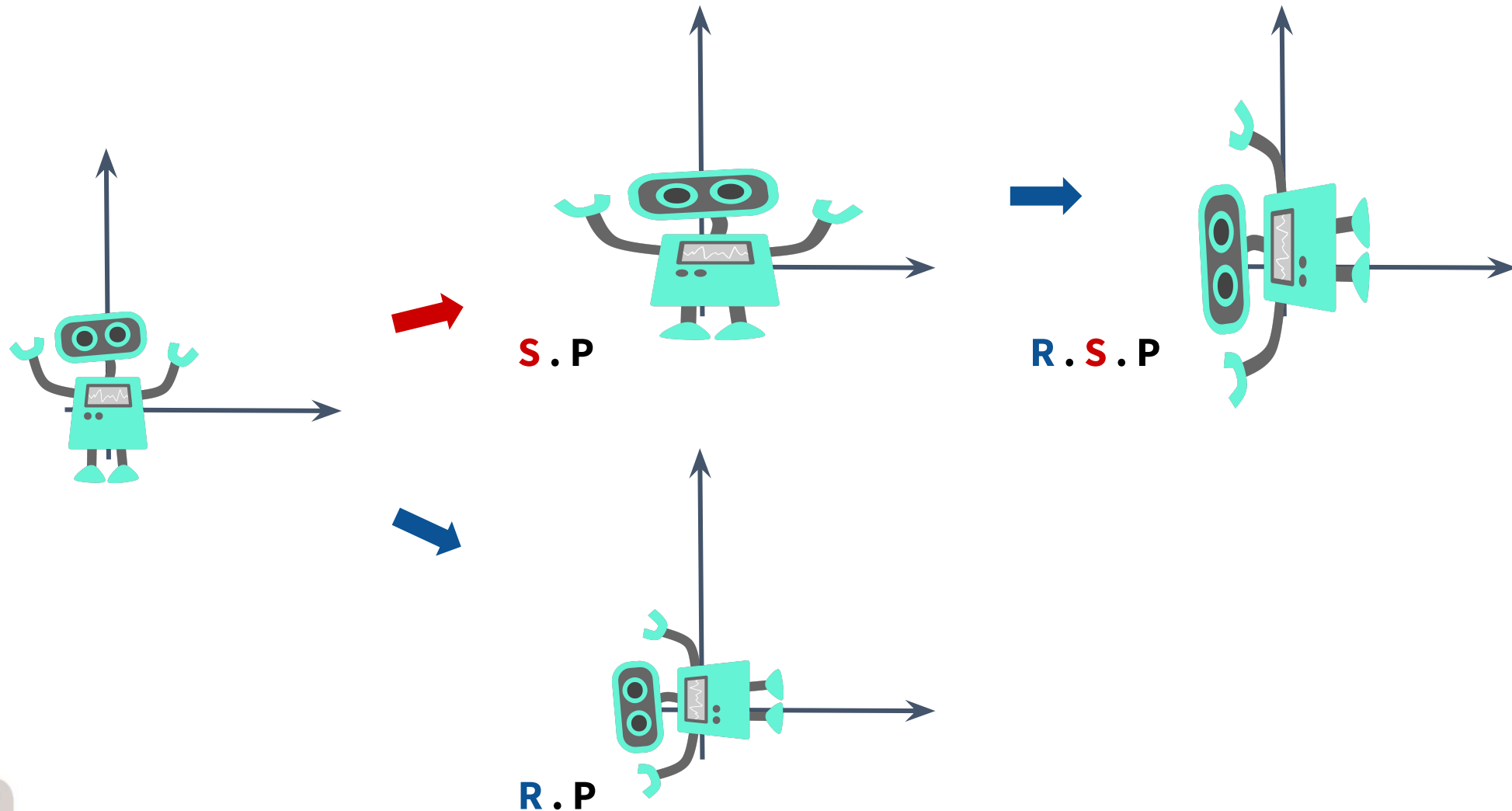
A ordem dos fatores altera o resultado



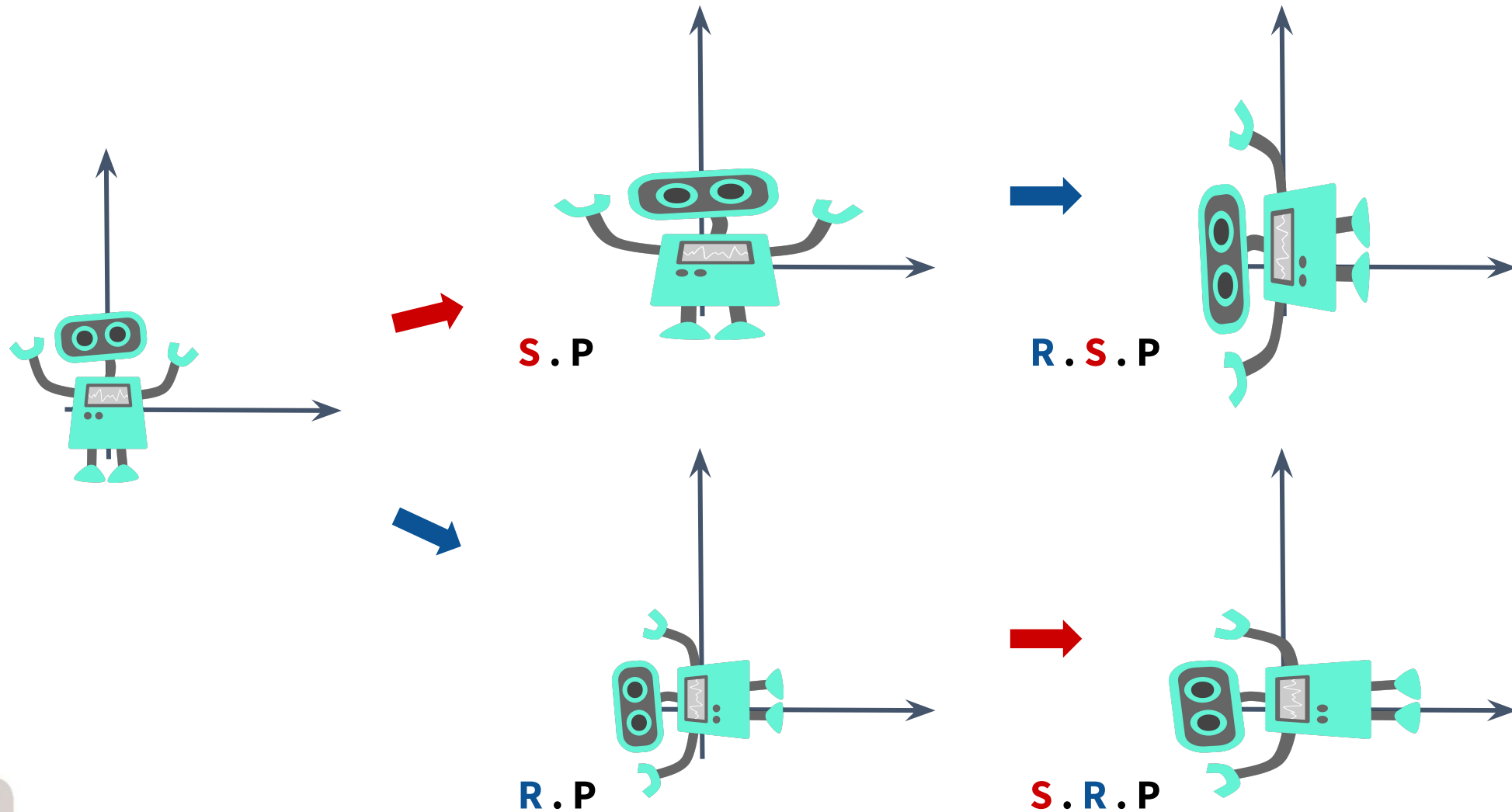
A ordem dos fatores altera o resultado



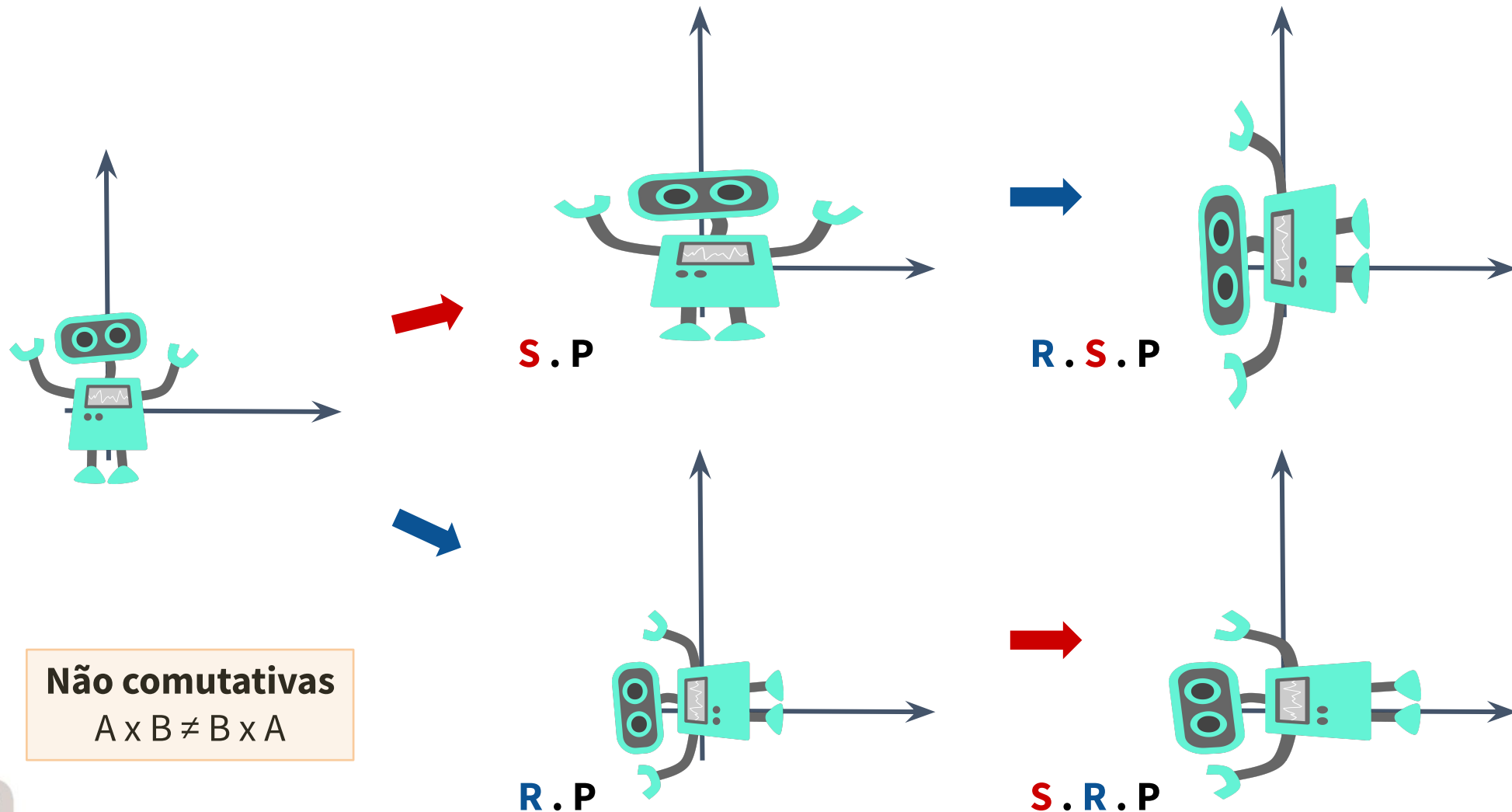
A ordem dos fatores altera o resultado



A ordem dos fatores altera o resultado



A ordem dos fatores altera o resultado



Exemplo

- Dado um quadrado bidimensional com vértices $P1 = (4, 4)$, $P2 = (6, 4)$, $P3 = (4, 6)$ e $P4 = (6, 6)$. Dê a matriz composta de transformação geométrica necessária para rotacioná-lo em 45° usando o centróide do quadrado como ponto de rotação (o centróide de um quadrado é a média de seus vértices em x e y)
- Dê também as coordenadas finais do quadrado após essa transformação
- **Dicas:** $\cos 45^\circ = \sin 45^\circ = \sqrt{2}/2$

Exemplo - solução

- Dado um quadrado bidimensional com vértices $P1 = (4, 4)$, $P2 = (6, 4)$, $P3 = (4, 6)$ e $P4 = (6, 6)$. Dê a matriz composta de transformação geométrica necessária para rotacioná-lo em 45° usando o centróide do quadrado como ponto de rotação

$$R(x_r, y_r, \theta) = R(5, 5, 45^\circ) = T(5, 5) \cdot R(45^\circ) \cdot T(-5, -5)$$

$$R(5, 5, 45^\circ) = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \sqrt{2}/2 & -\sqrt{2}/2 & 0 \\ \sqrt{2}/2 & \sqrt{2}/2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -5 \\ 0 & 1 & -5 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R(5, 5, 45^\circ) = \begin{bmatrix} 0.7071 & -0.7071 & 5 \\ 0.7071 & 0.7071 & -2.0710 \\ 0 & 0 & 1 \end{bmatrix}$$

- Dicas:** $\cos(45^\circ) = \sin(45^\circ) = \sqrt{2}/2$

Exemplo - solução

- Dê também as coordenadas finais do quadrado após essa transformação
 - Lembrando: $P1 = (x_1, y_1) = (4, 4)$

$$P1' = R(5, 5, 45^\circ) \cdot P1$$
$$\begin{bmatrix} x'_1 \\ y'_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.7071 & -0.7071 & 5 \\ 0.7071 & 0.7071 & -2.0710 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 4 \\ 1 \end{bmatrix}$$
$$\begin{bmatrix} x'_1 \\ y'_1 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 5 \\ 3.5857 \\ 1 \end{bmatrix}$$

Exemplo - solução

- Dê também as coordenadas finais do quadrado após essa transformação
 - Lembrando: $P2 = (x_2, y_2) = (6, 4)$

$$P2' = R(5, 5, 45^\circ) \cdot P2$$
$$\begin{bmatrix} x'_2 \\ y'_2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.7071 & -0.7071 & 5 \\ 0.7071 & 0.7071 & -2.0710 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 6 \\ 4 \\ 1 \end{bmatrix}$$
$$\begin{bmatrix} x'_2 \\ y'_2 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 6.4142 \\ 5 \\ 1 \end{bmatrix}$$

Exemplo - solução

- Dê também as coordenadas finais do quadrado após essa transformação
 - Lembrando: $P3 = (x_3, y_3) = (4, 6)$

$$P3' = R(5, 5, 45^\circ) \cdot P3$$
$$\begin{bmatrix} x'_3 \\ y'_3 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.7071 & -0.7071 & 5 \\ 0.7071 & 0.7071 & -2.0710 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 6 \\ 1 \end{bmatrix}$$
$$\begin{bmatrix} x'_3 \\ y'_3 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 3.5857 \\ 5 \\ 1 \end{bmatrix}$$

Exemplo - solução

- Dê também as coordenadas finais do quadrado após essa transformação
 - Lembrando: $P4 = (x_4, y_4) = (6, 6)$

$$P4' = R(5, 5, 45^\circ) \cdot P4$$
$$\begin{bmatrix} x'_4 \\ y'_4 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.7071 & -0.7071 & 5 \\ 0.7071 & 0.7071 & -2.0710 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 6 \\ 6 \\ 1 \end{bmatrix}$$
$$\begin{bmatrix} x'_4 \\ y'_4 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 5 \\ 6.4142 \\ 1 \end{bmatrix}$$

Exemplo - solução

- Dê também as coordenadas finais do quadrado após essa transformação

$$P1' \approx (5, 3.5857)$$

$$P2' \approx (6.4142, 5)$$

$$P3' \approx (3.5857, 5)$$

$$P4' \approx (5, 6.4142)$$

06

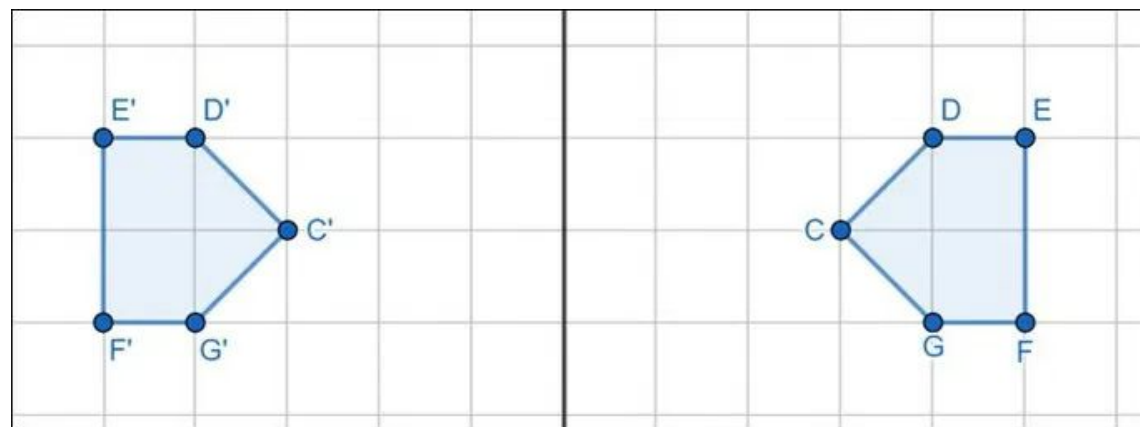
Outras transformações 2D

Reflexão

Cisalhamento

Reflexão

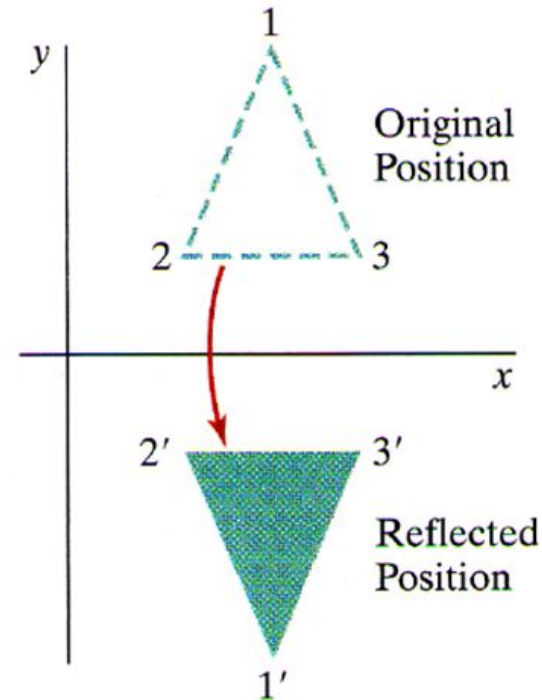
- Espelha-se as coordenadas de um objeto relativo a um eixo de reflexão, rotacionando em um ângulo de 180°



Reflexão vertical

- Consiste em refletir um objeto em torno do eixo x, ou seja, espelhá-lo em relação ao eixo horizontal. Nesse processo, a coordenada y de cada ponto é invertida, enquanto a coordenada x permanece inalterada

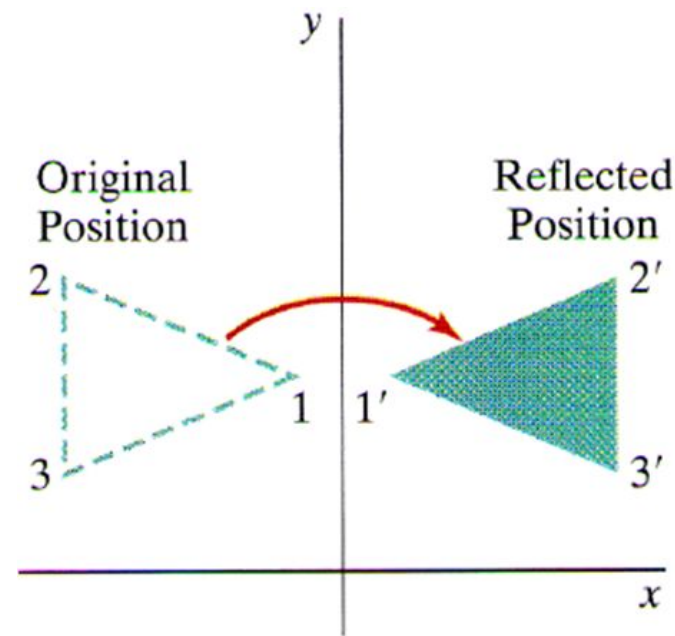
$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Reflexão horizontal

- Consiste em refletir um objeto em torno do eixo y, ou seja, espelhá-lo em relação ao eixo vertical. Nesse processo, a coordenada x de cada ponto é invertida, enquanto a coordenada y permanece inalterada

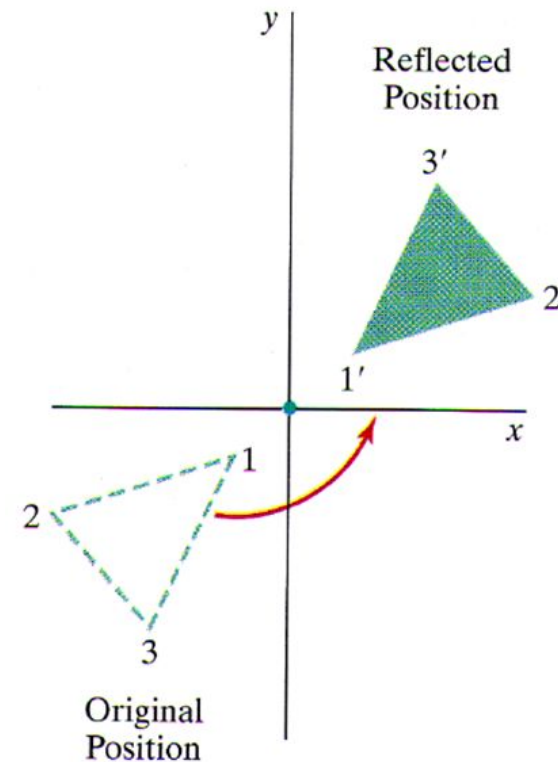
$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Reflexão no ponto de origem

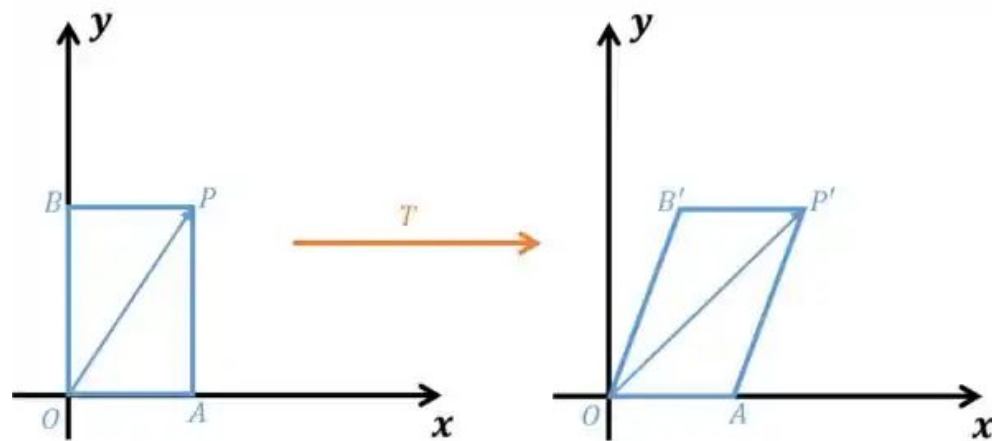
- Resulta em uma inversão do objeto tanto ao longo do eixo horizontal quanto do eixo vertical

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Cisalhamento

- Distorce o formato do objeto na direção do eixo x ou y



Cisalhamento horizontal

- Desloca os pontos de um objeto ao longo do eixo x, enquanto a coordenada y permanece inalterada

$$\begin{aligned}x' &= x + sh_x \cdot y \\ y' &= y\end{aligned}$$

- Notação matricial

$$\begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Cisalhamento vertical

- Desloca os pontos de um objeto ao longo do eixo y, enquanto a coordenada x permanece inalterada

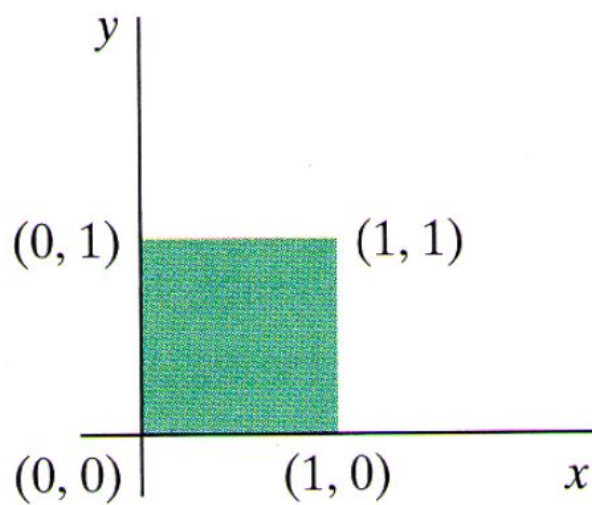
$$\begin{aligned}x' &= x \\ y' &= y + sh_y \cdot x\end{aligned}$$

- Notação matricial

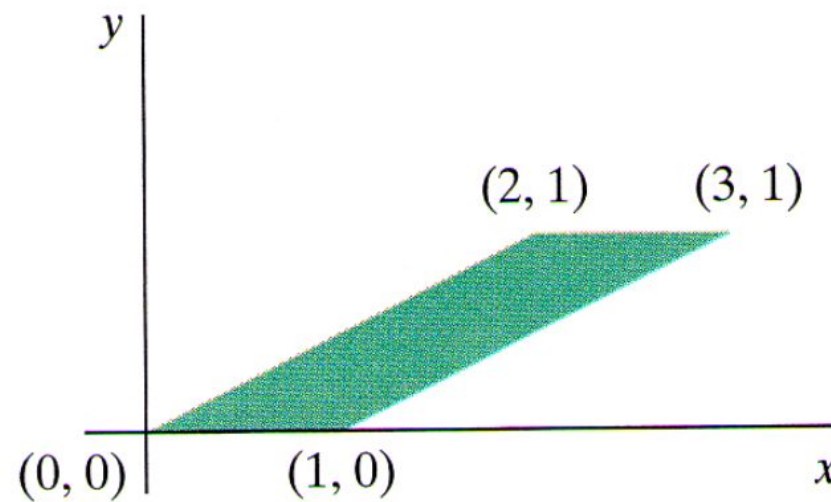
$$\begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Exemplo

- Convertendo um quadrado em um paralelogramo usando um cisalhamento horizontal (na direção x) com $sh_x = 2$



(a)



(b)

$$\begin{aligned}x' &= x + sh_x \cdot y \\ y' &= y\end{aligned}$$

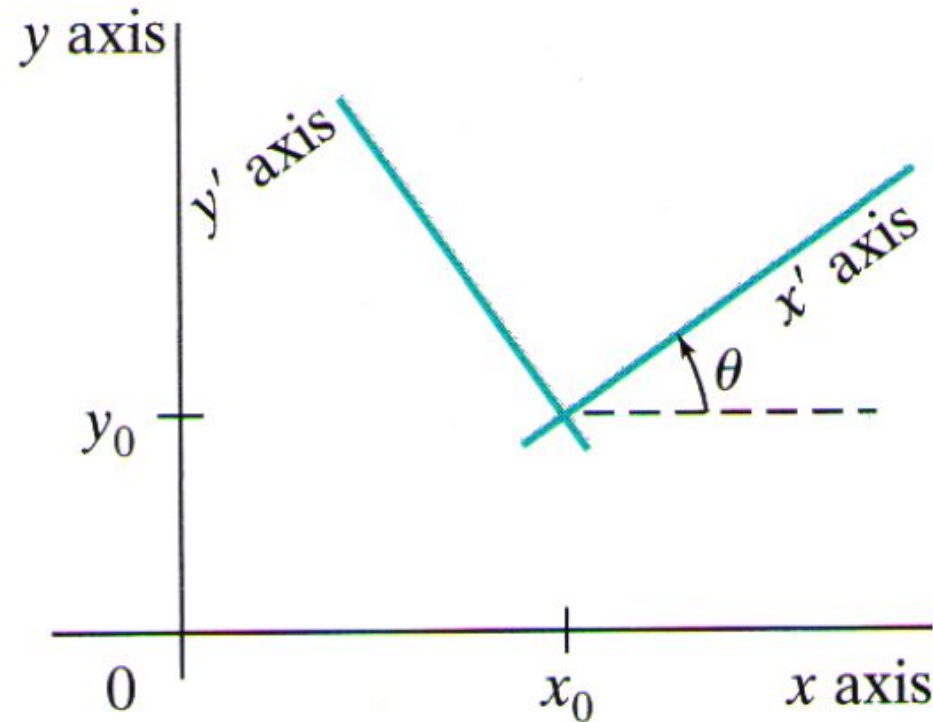
07

Transformações entre sistemas de coordenadas

Aplicação
Exemplos

Transformações entre sistemas de coordenadas

- Aplicações de computação gráfica envolvem a transformação de um sistema de coordenadas em outro em vários estágios do processamento da cena



Transformações entre sistemas de coordenadas

- Para se transformar um sistema de coordenadas xy em outro $x'y'$
 - 1) Translade (x_0, y_0) para a origem $(0, 0)$
 - 2) Rotacione em $-\theta$

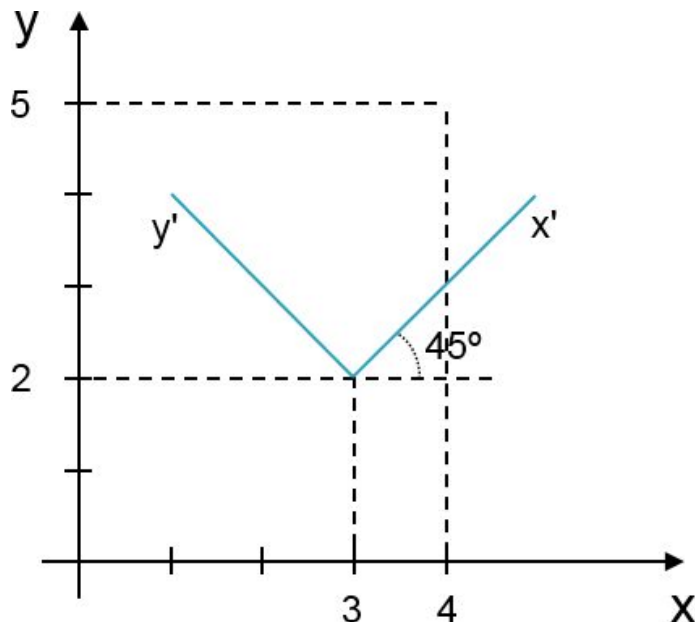
$$M_{xy,x'y'} = R(-\theta) \cdot T(-x_0, -y_0)$$

- 3) Aplica a matriz composta no ponto indexado no sistema de coordenadas xy
- Essa transformação nos dá a geometria da cena em relação ao novo sistema de referência $x'y'$ (a cena é a mesma)

Transformações entre sistemas de coordenadas

- **Exemplo - solução**

- Calcular a matriz de transformação de xy para $x'y'$ e as coordenadas finais do ponto P no sistema destino (P')
 - Dicas: $\theta = 45^\circ$; $(x_0, y_0) = (3, 2)$; $P = (4, 5)$; e $\sin 45^\circ = \cos 45^\circ = \sqrt{2}/2$



$$\begin{aligned} M_{xy,x'y'} &= R(-45^\circ) \cdot T(-3, -2) = \\ &= \begin{bmatrix} \cos 45^\circ & \sin 45^\circ & 0 \\ -\sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -3 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix} = \\ &= \begin{bmatrix} \sqrt{2}/2 & \sqrt{2}/2 & 0 \\ -\sqrt{2}/2 & \sqrt{2}/2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -3 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Transformações entre sistemas de coordenadas

- **Exemplo - solução**

- Calcular a matriz de transformação de xy para $x'y'$ e as coordenadas finais do ponto P no sistema destino (P')
 - Dicas: $\theta = 45^\circ$; $(x_0, y_0) = (3, 2)$; $P = (4, 5)$; e $\sin 45^\circ = \cos 45^\circ = \sqrt{2}/2$

$$M_{xy,x'y'} = R(-45^\circ) \cdot T(-3, -2) = \begin{bmatrix} \sqrt{2}/2 & \sqrt{2}/2 & -5\sqrt{2}/2 \\ -\sqrt{2}/2 & \sqrt{2}/2 & \sqrt{2}/2 \\ 0 & 0 & 1 \end{bmatrix}$$

$$P' = M_{xy,x'y'} \cdot P = \begin{bmatrix} \sqrt{2}/2 & \sqrt{2}/2 & -5\sqrt{2}/2 \\ -\sqrt{2}/2 & \sqrt{2}/2 & \sqrt{2}/2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix}$$

$$P' = \begin{bmatrix} 4\sqrt{2}/2 \\ 2\sqrt{2}/2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2.8284 \\ 1.4142 \\ 1 \end{bmatrix} = (2.8284, 1.4142)$$

Transformações entre sistemas de coordenadas

- **Exemplo - solução**

- Calcular a matriz de transformação de xy para $x'y'$ e as coordenadas finais do ponto P no sistema destino (P')
 - Dicas: $\theta = 45^\circ$; $(x_0, y_0) = (3, 2)$; $P = (4, 5)$; e $\sin 45^\circ = \cos 45^\circ = \sqrt{2}/2$

$$M_{xy,x'y'} = R(-45^\circ) \cdot T(-3, -2) = \begin{bmatrix} \sqrt{2}/2 & \sqrt{2}/2 & -5\sqrt{2}/2 \\ -\sqrt{2}/2 & \sqrt{2}/2 & \sqrt{2}/2 \\ 0 & 0 & 1 \end{bmatrix}$$

$$P' = M_{xy,x'y'} \cdot P = \begin{bmatrix} \sqrt{2}/2 & \sqrt{2}/2 & -5\sqrt{2}/2 \\ -\sqrt{2}/2 & \sqrt{2}/2 & \sqrt{2}/2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix}$$

$$P' = \begin{bmatrix} 4\sqrt{2}/2 \\ 2\sqrt{2}/2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2.8284 \\ 1.4142 \\ 1 \end{bmatrix} = (2.8284, 1.4142)$$

Este é o **1º método** para resolver o problema. Entretanto, o θ geralmente **não** é fornecido!

Transformações entre sistemas de coordenadas

- Uma propriedade importante da matriz de transformação é que a submatriz de rotação 2 x 2 é ortonormal

$$\begin{bmatrix} r_{xx} & r_{xy} & tr_x \\ r_{yx} & r_{yy} & tr_y \\ 0 & 0 & 1 \end{bmatrix}$$

Ortonormais são vetores (eixos) que precisam ser

- **Unitários** (norma tem que dar igual a 1)
- **Ortogonais** (tem que fazer um ângulo de 90° entre si)

- Isto é, usar cada linha (r_{xx}, r_{xy}) e (r_{yx}, r_{yy}) como eixo forma um conjunto de vetores unitários ortogonais (ortonormais)

$$\sqrt{r_{xx}^2 + r_{xy}^2} = \sqrt{r_{yx}^2 + r_{yy}^2} = 1$$

$$(r_{xx}, r_{xy}) \cdot (r_{yx}, r_{yy}) = r_{xx} \cdot r_{yx} + r_{xy} \cdot r_{yy} = 0 = \cos 90^\circ$$

Transformações entre sistemas de coordenadas

- Uma propriedade importante da matriz de transformação é que a submatriz de rotação 2 x 2 é ortonormal

$$\begin{bmatrix} r_{xx} & r_{xy} & tr_x \\ r_{yx} & r_{yy} & tr_y \\ 0 & 0 & 1 \end{bmatrix}$$

Ortonormais são vetores (eixos) que precisam ser

- **Unitários** (norma tem que dar igual a 1)
- **Ortogonais** (tem que fazer um ângulo de 90° entre si)

- Isto é, usar cada linha (r_{xx}, r_{xy}) e (r_{yx}, r_{yy}) como eixo forma um conjunto de vetores unitários ortogonais (ortonormais)

$$\sqrt{r_{xx}^2 + r_{xy}^2} = \sqrt{r_{yx}^2 + r_{yy}^2} = 1$$

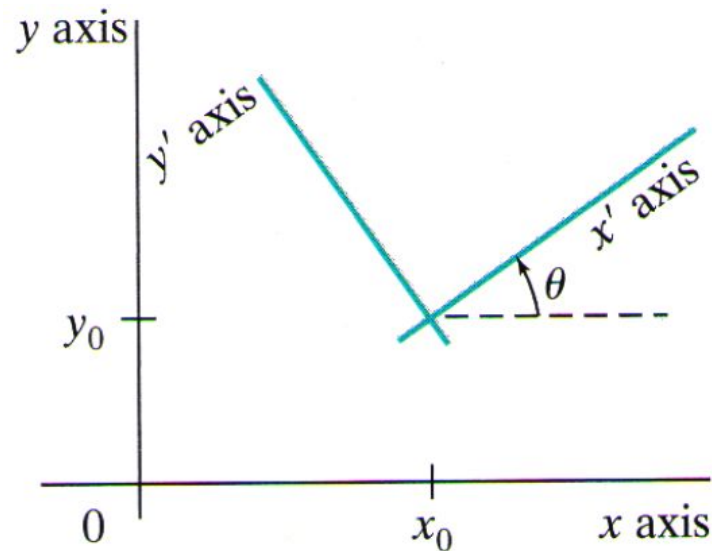
$$(r_{xx}, r_{xy}) \cdot (r_{yx}, r_{yy}) = r_{xx} \cdot r_{yx} + r_{xy} \cdot r_{yy} = 0 = \cos 90^\circ$$

Propriedade do produto escalar

- O produto escalar de dois vetores unitários é igual ao cosseno do menor ângulo entre eles

Transformações entre sistemas de coordenadas

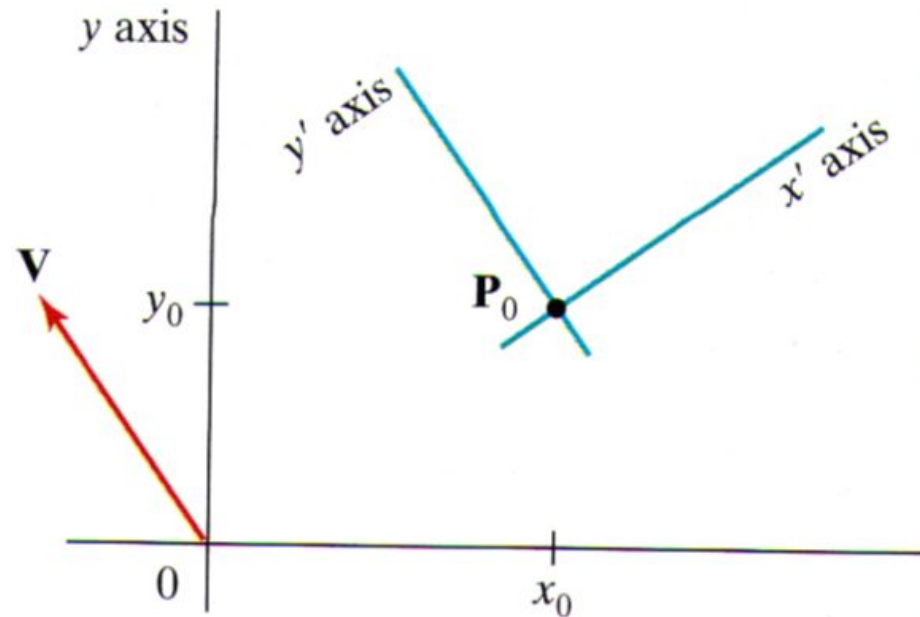
- Portanto, qualquer matriz de rotação pode ser expressa por um conjunto de vetores ortonormais. Assim, sabendo que a submatriz responsável pela rotação é ortonormal, é possível derivar um método alternativo para transformar xy em x', y'



2º método para resolver o problema!

Transformações entre sistemas de coordenadas

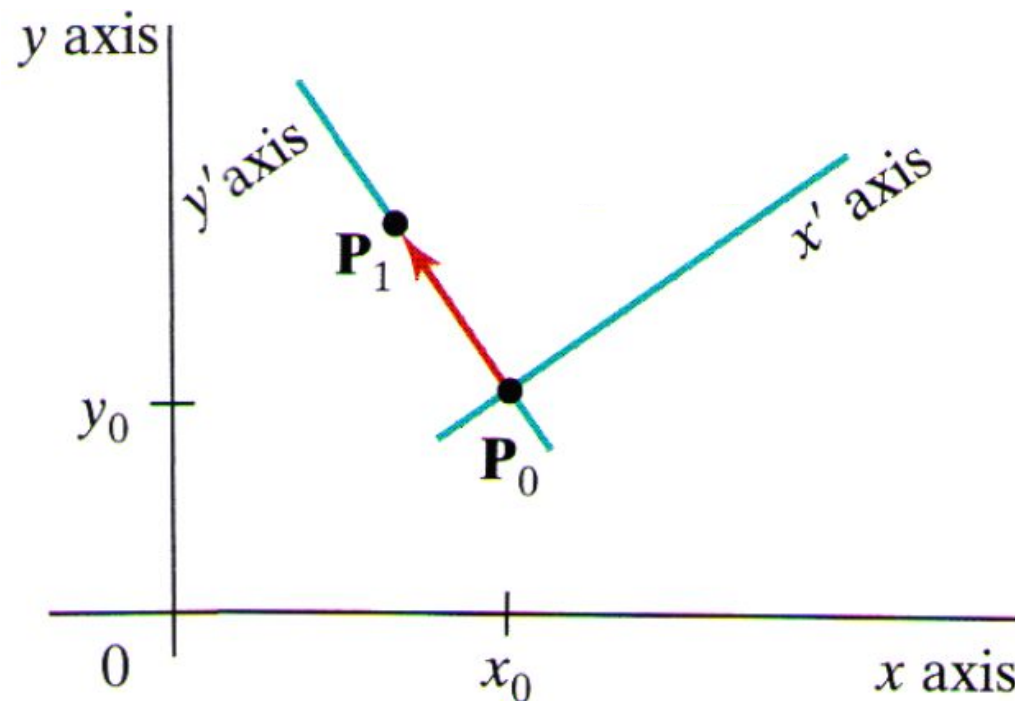
- Para isso, precisamos encontrar o conjunto de vetores unitários e ortogonais por meio dos eixos de coordenadas $x'y'$ para montar a matriz de rotação sem o θ



Transformações entre sistemas de coordenadas

- É possível especificar v com base em P_0 e P_1

$$v = P_1 - P_0 = (V_x, V_y)$$



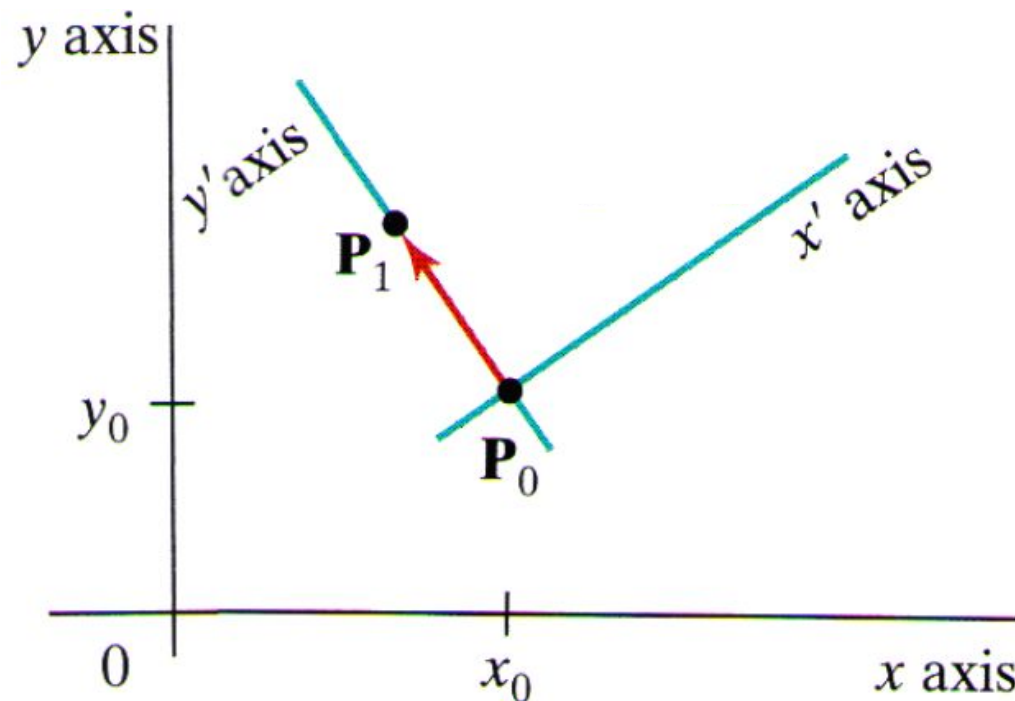
Transformações entre sistemas de coordenadas

- É possível especificar v com base em P_0 e P_1

$$v = P_1 - P_0 = (V_x, V_y)$$

Coordenadas em maiúsculas

- Se calcular a norma não há garantia de que é igual a 1



Transformações entre sistemas de coordenadas

- Para tornar o vetor V unitário, divide cada coordenada do V pela norma

$$v = \frac{V}{|V|} = \frac{(V_x, V_y)}{|(V_x, V_y)|} = (v_x, v_y)$$

- E podemos obter o vetor unitário u ortogonal a v na direção do eixo x'

$$u = (v_y, -v_x) = (u_x, u_y)$$

Transformações entre sistemas de coordenadas

- Como qualquer matriz de rotação pode ser expressa por um conjunto de vetores ortonormais, então podemos escrever a matriz de rotação que faz $x'y'$ coincidir com xy como

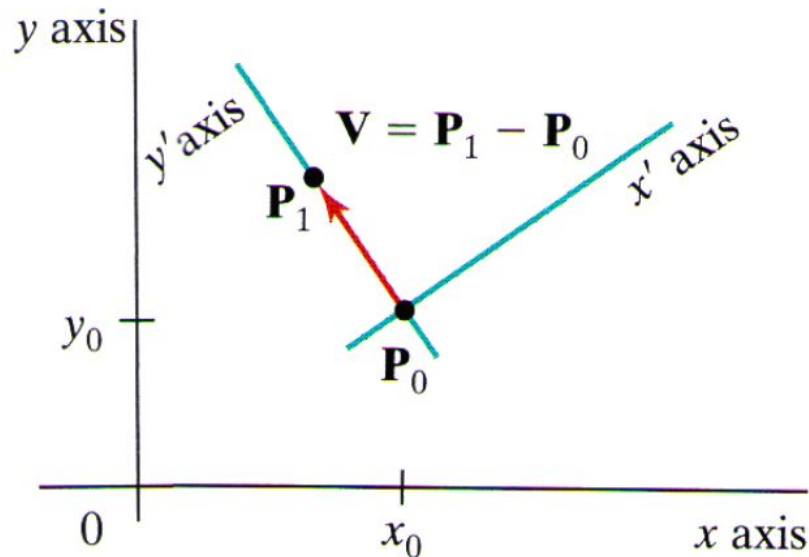
$$R = \begin{bmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} v_y & -v_x & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Observe, que a **translação** para alinhar a origem dos dois sistemas de coordenadas ainda é necessária

Transformações entre sistemas de coordenadas

- **Exemplo - solução**

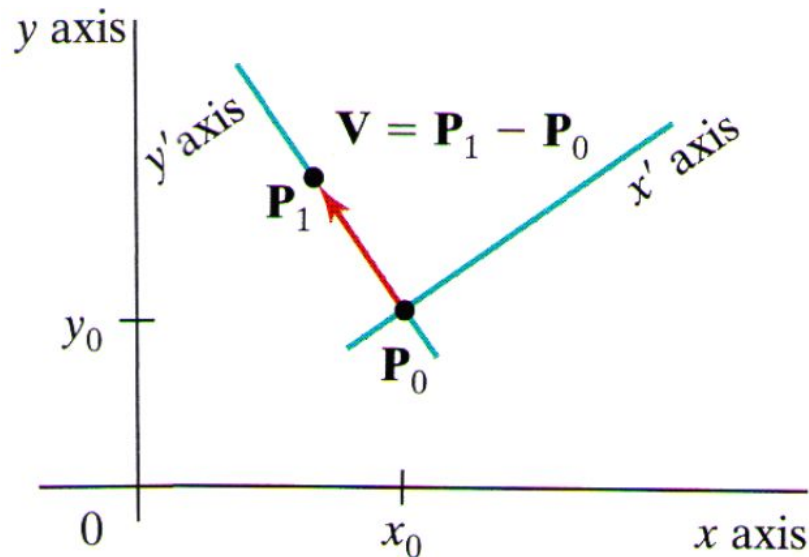
- Calcular a matriz de transformação de xy para $x'y'$ e as coordenadas finais do ponto P no sistema destino (P')
 - Dicas: $(x_0, y_0) = (4, 3)$; $(x_1, y_1) = (3, 4)$; e $P = (-1, 5)$



Transformações entre sistemas de coordenadas

- **Exemplo - solução**

- Calcular a matriz de transformação de xy para $x'y'$ e as coordenadas finais do ponto P no sistema destino (P')
 - Dicas: $(x_0, y_0) = (4, 3)$; $(x_1, y_1) = (3, 4)$; e $P = (-1, 5)$

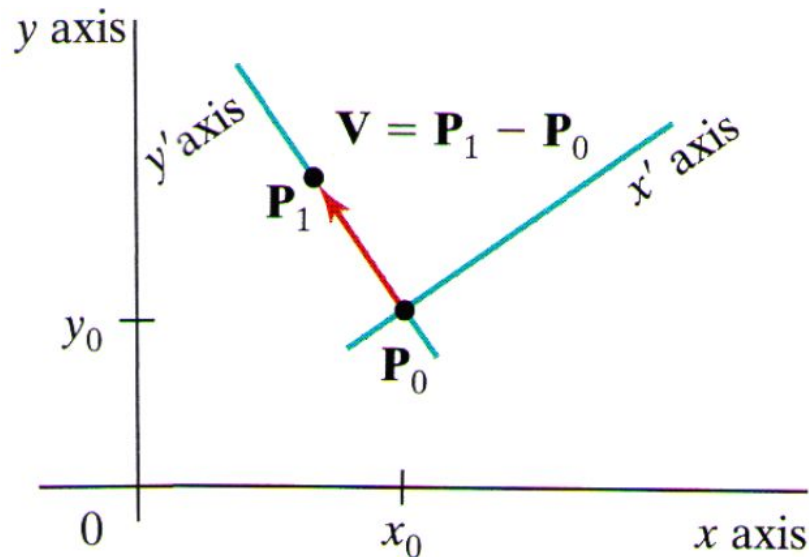


$$\begin{aligned}v &= \frac{P_1 - P_0}{|P_1 - P_0|} = \frac{(x_1, y_1) - (x_0, y_0)}{|(x_1, y_1) - (x_0, y_0)|} \\v &= \frac{(3, 4) - (4, 3)}{|(3, 4) - (4, 3)|} = \frac{(-1, 1)}{|(-1, 1)|} \\v &= \frac{(-1, 1)}{\sqrt{(-1)^2 + (1)^2}} = \frac{(-1, 1)}{\sqrt{2}} \\v &= \left(\frac{-1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right) \\v_x &= \frac{-1}{\sqrt{2}} \quad v_y = \frac{1}{\sqrt{2}}\end{aligned}$$

Transformações entre sistemas de coordenadas

- **Exemplo - solução**

- Calcular a matriz de transformação de xy para $x'y'$ e as coordenadas finais do ponto P no sistema destino (P')
 - Dicas: $(x_0, y_0) = (4, 3)$; $(x_1, y_1) = (3, 4)$; e $P = (-1, 5)$



$$R = \begin{bmatrix} v_y & -v_x & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{-1}{\sqrt{2}} & 0 \\ \frac{-1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

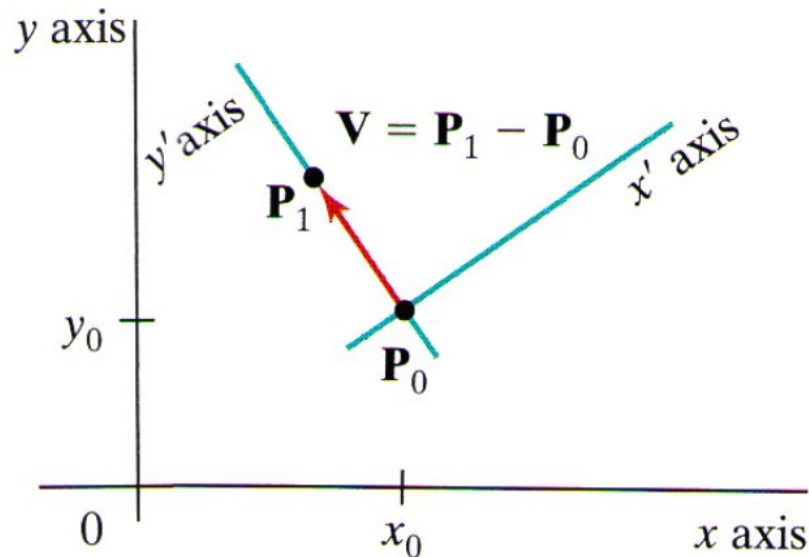
$$R = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$T = \begin{bmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -4 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix}$$

Transformações entre sistemas de coordenadas

- **Exemplo - solução**

- Calcular a matriz de transformação de xy para $x'y'$ e as coordenadas finais do ponto P no sistema destino (P')
 - Dicas: $(x_0, y_0) = (4, 3)$; $(x_1, y_1) = (3, 4)$; e $P = (-1, 5)$



$$M_{xy,x'y'} = R \cdot T$$

$$M_{xy,x'y'} = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -4 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix}$$

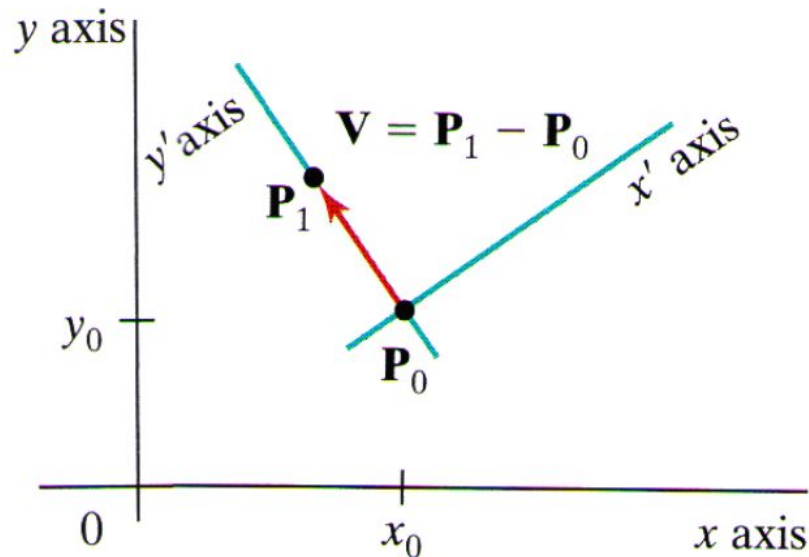
$$M_{xy,x'y'} = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & -\frac{7}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 & 1 \end{bmatrix}$$

$$P' = M_{xy,x'y'} \cdot P$$

Transformações entre sistemas de coordenadas

- **Exemplo - solução**

- Calcular a matriz de transformação de xy para $x'y'$ e as coordenadas finais do ponto P no sistema destino (P')
 - Dicas: $(x_0, y_0) = (4, 3)$; $(x_1, y_1) = (3, 4)$; e $P = (-1, 5)$



$$P' = M_{xy, x'y'} \cdot P$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & -\frac{7}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 5 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -\frac{3}{\sqrt{2}} \\ \frac{7}{\sqrt{2}} \\ 1 \end{bmatrix}$$

$$(x', y') = (-2.1213, 4.9497)$$

Transformações 2D em OpenGL

Translação

Rotação

Escala

Operações sobre matrizes em OpenGL

Exemplos

Translação

- O seguinte comando cria uma matriz 4 x 4 de translação

```
glTranslate*(tx, ty, tz);
```

- Parâmetros
 - O * pode ser substituído por f (float) ou d (double)
 - tx, ty e tz são os offsets de translação nos eixos x, y e z, respectivamente
 - Para translação em 2D, coloque tz=0

- **Exemplo**

```
glTranslatef(25.0, -10.0, 0.0);
```

Rotação

- O seguinte comando cria uma matriz 4 x 4 de rotação

```
glRotate*(theta, vx, vy, vz);
```

- Parâmetros
 - O * pode ser substituído por f (float) ou d (double)
 - theta é o ângulo de rotação em graus
 - vx, vy e vz definem a orientação do eixo de rotação, que passa pelo ponto de origem (0, 0, 0)
 - Para a rotação em 2D, coloque vx=0, vy=0 e vz=1

- **Exemplo**

```
glRotatef(90.0, 0.0, 0.0, 1.0);
```

Escala

- O seguinte comando cria uma matriz 4 x 4 de escala em relação a origem do sistema de coordenadas

```
glScale*(sx, sy, sz);
```

- Parâmetros
 - O * pode ser substituído por f (float) ou d (double)
 - sx, sy e sz são os fatores de escalonamento
 - Para a escala 2D, coloque sz=1
 - Valores negativos geram reflexão

- **Exemplo**

```
glScalef(2.0, 0.2, 1.0);
```

Operações sobre matrizes em OpenGL

- O seguinte comando concatena a matriz especificada com a matriz atual

```
glMultMatrix* (matrix_param) ;
```

- Parâmetros
 - O * pode ser substituído por f (float) ou d (double)
 - matrix_param é vetor com 16 elementos que representa a transformação desejada

- **Exemplo**

```
float cisalhamento_x[] = {1, shx, 0, 0,  
                           0, 1, 0, 0,  
                           0, 0, 1, 0,  
                           0, 0, 0, 1};  
  
glMultMatrixf (cisalhamento_x);
```

Operações sobre matrizes em OpenGL

- Antes de usar qualquer comando para uma transformação geométrica, é necessário dizer que estaremos utilizando matrizes para transformações geométricas
- Especifique isso, usando

```
glMatrixMode (GL_MODELVIEW) ;
```

Transformações 2D em OpenGL

- Exemplo 1 – Escala com ponto fixo

```
#include<windows.h>
#include <stdio.h>
#include <GL/freeglut.h>

int init(void);
void display(void);
void desenhaCasa(void);
int main(int argc, char** argv);

int main(int argc, char** argv) {
    glutInit(&argc,argv);           //inicializa o GLUT
    glutInitDisplayMode(GLUT_SINGLE| GLUT_RGB); //configura o modo de display
    glutInitWindowPosition(200,0);  //posição da janela
    glutInitWindowSize(400,300);    //configura a largura e altura da janela de exibição
    glutCreateWindow("Escala com Ponto Fixo"); //cria a janela de exibição

    init();                         //executa função de inicialização
    glutDisplayFunc(display);       //estabelece a função "display" como a função callback de exibição
    glutMainLoop();                //mostre tudo e espere
    return 0;
}
//continua...
```

Transformações 2D em OpenGL

- Exemplo 1 – Escala com ponto fixo

```
int init(void){
    glClearColor(1.0, 1.0, 1.0, 0.0);    //define a cor de fundo

    glMatrixMode(GL_PROJECTION);          //carrega a matriz de projeção
    gluOrtho2D(0,200,0,150);              /*define projeção ortogonal 2D que mapeia
    objetos da coordenada do mundo para coordenadas da tela*/
}

void desenhaCasa(void){
    glBegin(GL_POLYGON);                  //desenha uma casa
        glVertex2f(110,50);
        glVertex2f(110,70);
        glVertex2f(100,80);
        glVertex2f(90,70);
        glVertex2f(90,50);
    glEnd();
}
//continua...
```


Transformações 2D em OpenGL

- Exemplo 1 – Escala com ponto fixo

```
void display(void) {  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(1.0f, 0.0f, 0.0f);           //desenha objetos com a cor vermelha  
    glMatrixMode(GL_MODELVIEW);           //carrega a matriz de modelo  
  
    //utilizando o primeiro vértice da lista como ponto fixo  
    glTranslatef(110, 50, 0);              //move o ponto fixo para a posição original  
    glScalef(2.0, 2.0, 1.0);              //faz a escala  
    glTranslatef(-110, -50, 0);            //move o ponto fixo para a origem  
  
    desenhaCasa();  
  
    glFlush();                             //desenha os comandos não executados  
}
```


OpenGL: cumulativo

- Resultado antes e após a aplicação da transformação geométrica de escala. O OpenGL exibe apenas o resultado final, após a multiplicação das matrizes



OpenGL: cumulativo

- O método `display(...)` é chamado mais de uma vez (modificação do tamanho da janela) – o objeto é escalado novamente!

OpenGL: cumulativo

- Isso acontece porque o OpenGL é cumulativo, ou seja, a cada chamada de uma função que realiza transformações geométricas, irá multiplicar as matrizes de transformações pela matriz acumulada (referente às transformações anteriores)
- Para resolver isso, usamos a função **glLoadIdentity()** que serve para reiniciar a matriz acumulada para a matriz identidade. Isso descarta quaisquer transformações anteriores e permite que comece com uma matriz limpa antes de aplicar novas transformações, aos objetos renderizados. Em essência, ela é uma maneira de "zerar" as transformações e começar do ponto de partida original

Transformações 2D em OpenGL

- **Exemplo 2 – OpenGL: cumulativo**

- **Solução:** carregar a matriz identidade (glLoadIdentity())

```
void display(void) {  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(1.0f, 0.0f, 0.0f);           //desenha objetos com a cor vermelha  
    glMatrixMode(GL_MODELVIEW);           //carrega a matriz de modelo  
    glLoadIdentity();                     //carrega a matriz identidade  
  
    //utilizando o primeiro vértice da lista como ponto fixo  
    glTranslatef(110, 50, 0);              //move o ponto fixo para a posição original  
    glScalef(2.0, 2.0, 1.0);              //faz a escala  
    glTranslatef(-110, -50, 0);            //move o ponto fixo para a origem  
  
    desenhaCasa();  
  
    glFlush();                             //desenha os comandos não executados  
}
```

Transformações 2D em OpenGL

- **Exemplo 3 – OpenGL: alterando a ordem das transformações**
 - Primeiro rotaciono usando um ponto fixo, depois faço a translação

```
void display(void) {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 0.0f, 0.0f);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    glTranslatef(50, 0, 0);
    glTranslatef(110, 50, 0);
    glRotatef(90, 0, 0, 1);
    glTranslatef(-110, -50, 0);

    desenhaCasa();

    glFlush();
}
```

//desenha objetos com a cor vermelha
//carrega a matriz de modelo
//carrega a matriz identidade

//faz a translação
//move o ponto fixo para a posição original
//rotaciona
//move o ponto fixo para a origem

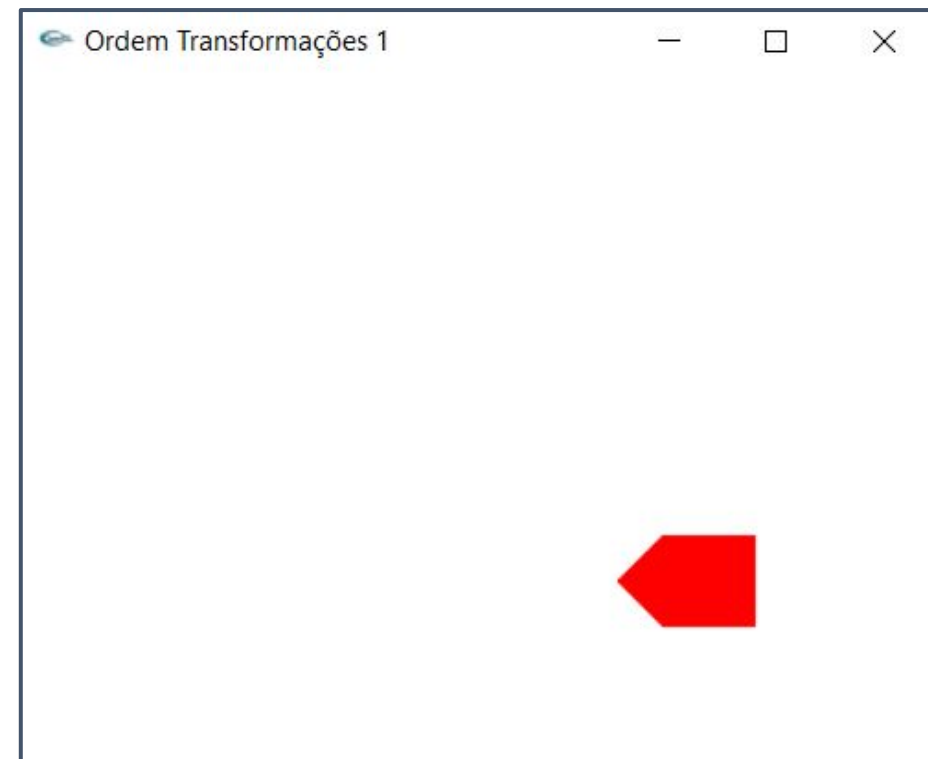
//desenha os comandos não executados

Transformações 2D em OpenGL

- **Exemplo 3 – OpenGL: alterando a ordem das transformações**
 - Primeiro rotaciono usando um ponto fixo, depois faço a translação



Antes



Depois

Transformações 2D em OpenGL

- **Exemplo 4 – OpenGL: alterando a ordem das transformações**
 - Primeiro faço a translação, depois rotaciono usando um ponto fixo

```
void display(void) {  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(1.0f, 0.0f, 0.0f);  
    glMatrixMode(GL_MODELVIEW);  
    glLoadIdentity();  
  
    glTranslatef(110, 50, 0);  
    glRotatef(90, 0, 0, 1);  
    glTranslatef(-110, -50, 0);  
    glTranslatef(50, 0, 0);  
  
    desenhaCasa();  
  
    glFlush();  
}
```

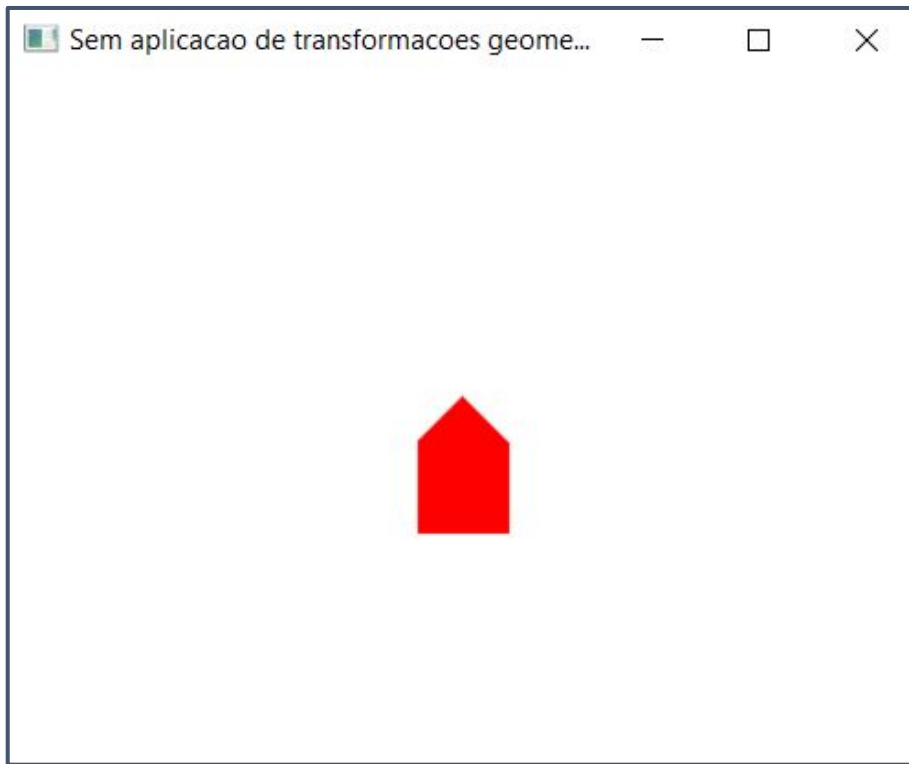
//desenha objetos com a cor vermelha
//carrega a matriz de modelo
//carrega a matriz identidade

//move o ponto fixo para a posição original
//rotaciona
//move o ponto fixo para a origem
//faz a translação

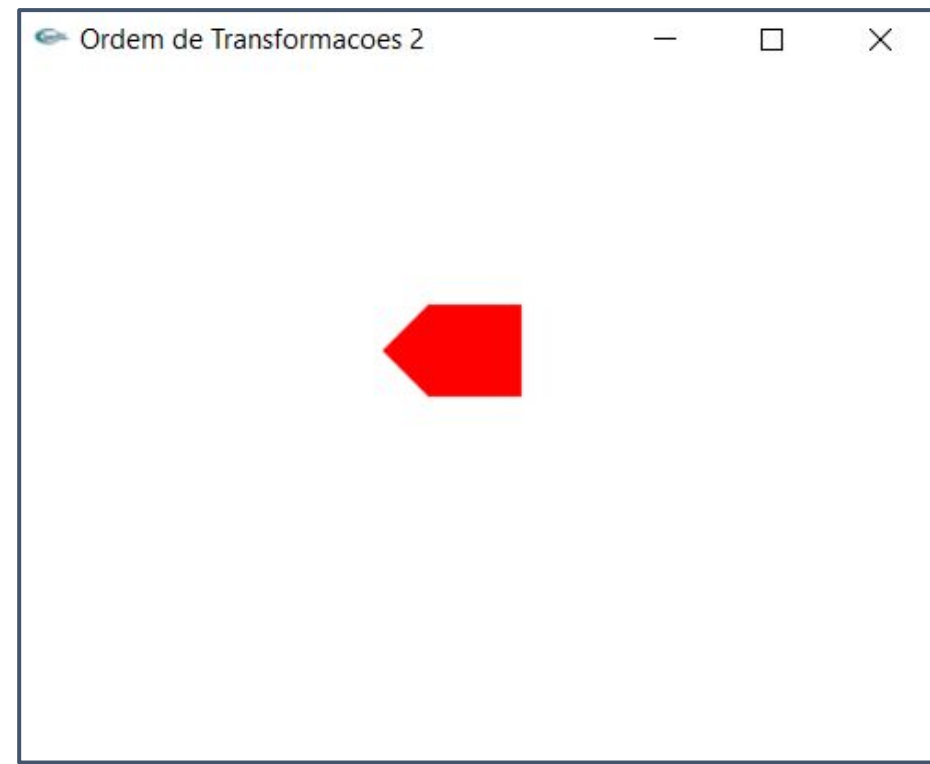
//desenha os comandos não executados

Transformações 2D em OpenGL

- **Exemplo 4 – OpenGL: alterando a ordem das transformações**
 - Primeiro faço a translação, depois rotaciono usando um ponto fixo



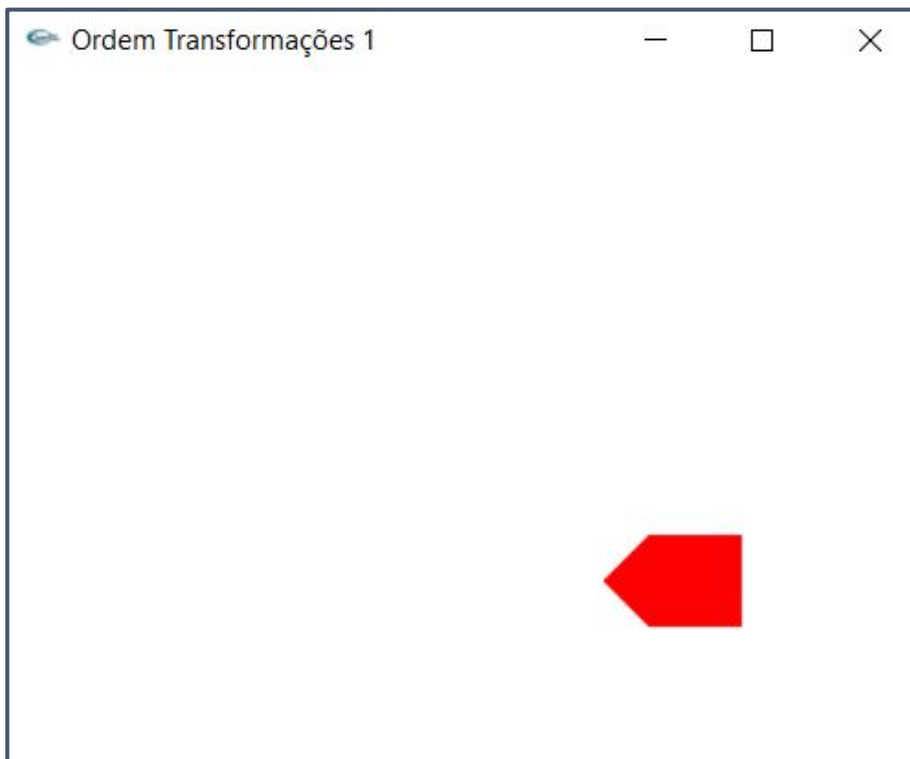
Antes



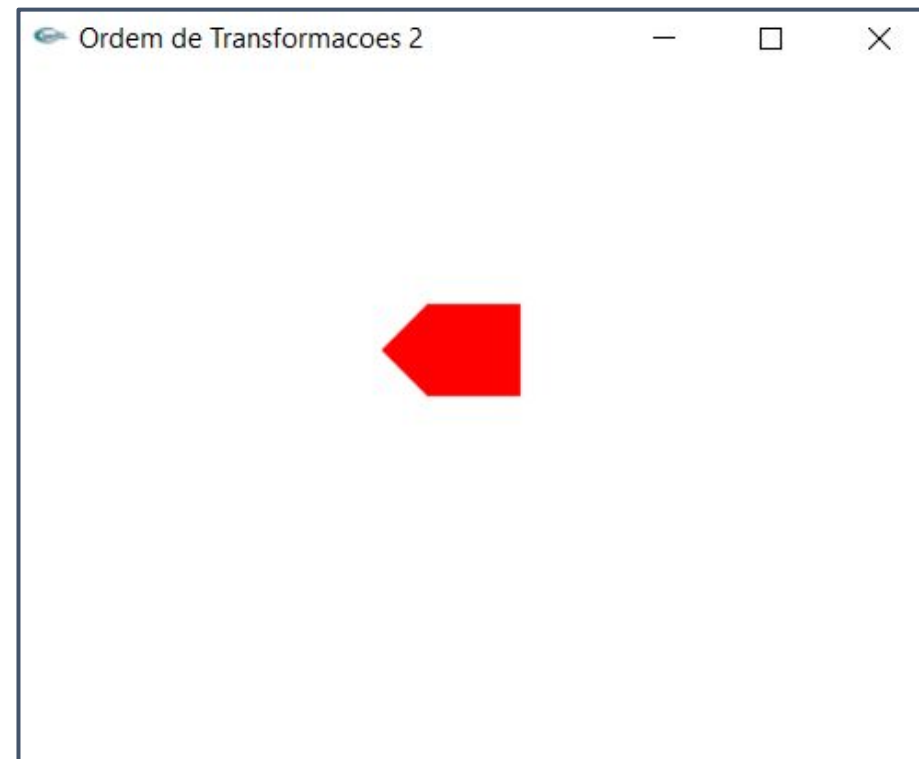
Depois

Transformações 2D em OpenGL

- **Exemplos 3 e 4 – OpenGL: alterando a ordem das transformações**



Exemplo 3



Exemplo 4

Transformações 2D em OpenGL

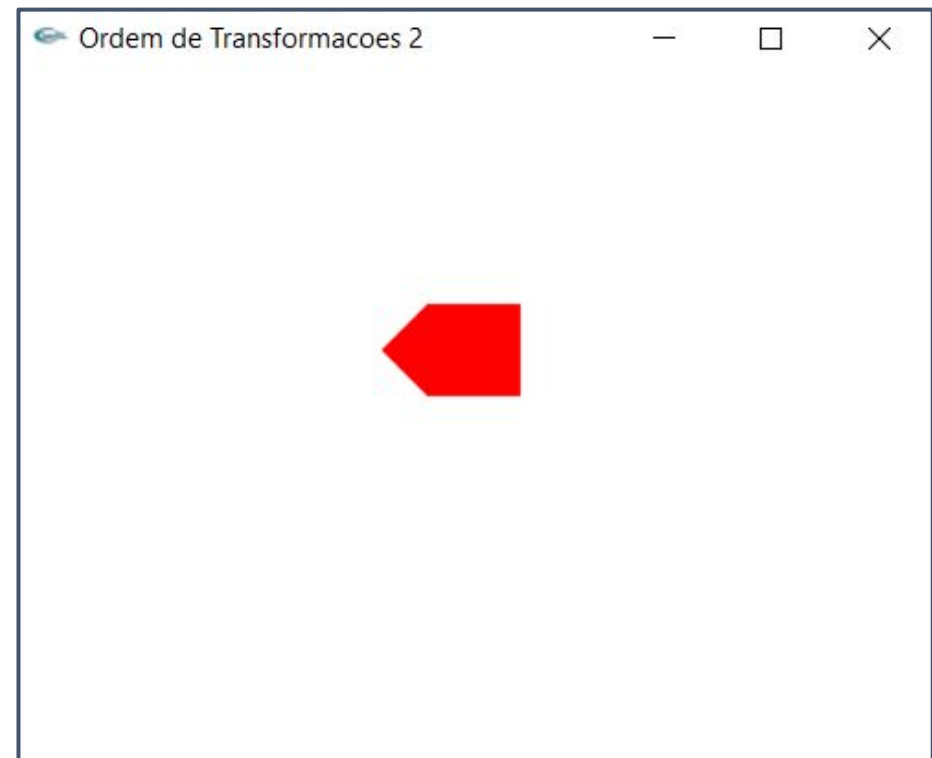
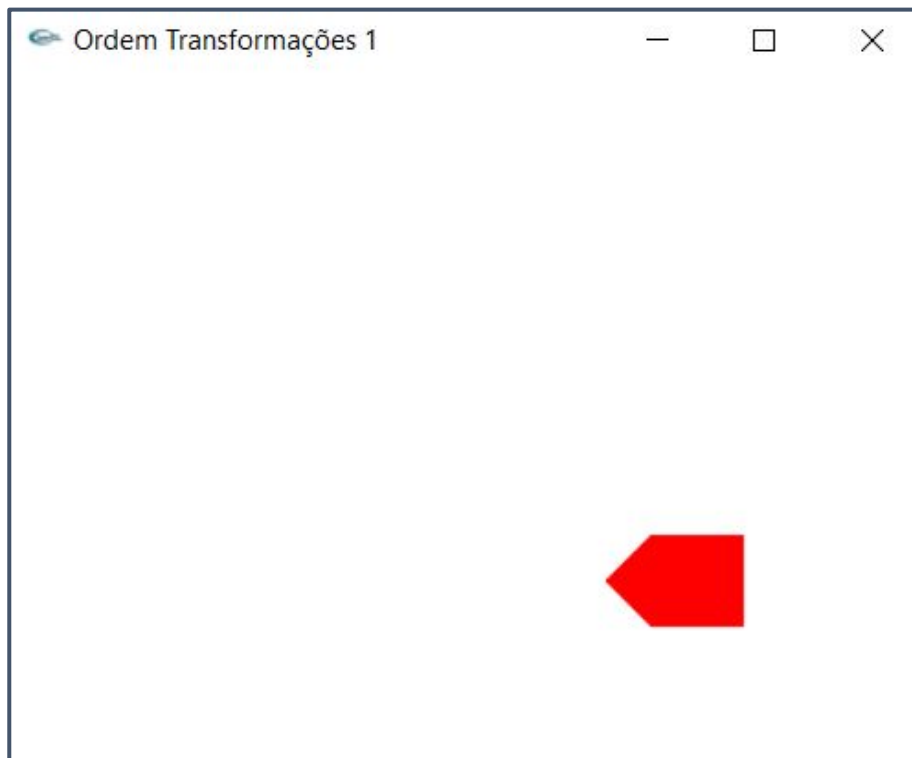
- **Exemplo 4 – OpenGL: alterando a ordem das transformações**
 - Primeiro faço a translação, depois rotaciono usando um ponto fixo

```
void display(void) {  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(1.0f, 0.0f, 0.0f);           //desenha objetos com a cor vermelha  
    glMatrixMode(GL_MODELVIEW);           //carrega a matriz de modelo  
    glLoadIdentity();                     //carrega a matriz identidade  
  
    glTranslatef(110, 50, 0);              para a posição original  
    glRotatef(90, 0, 0, 1);               para a origem  
    glTranslatef(-110, -50, 0);  
    glTranslatef(50, 0, 0);  
  
    desenhaCasa();  
  
    glFlush();                             //desenha os comandos não executados  
}
```

Os valores de x e y foram alterados na translação. Portanto, não é mais -110, -50 um dos vértices

OpenGL: ordem das transformações

- A ordem das transformações e valores incorretos podem gerar resultados completamente diferentes!



Transformações 2D em OpenGL

- Exemplo 5 – translada (em x) apenas um dos dois objetos

```
int main(int argc, char** argv){  
    glutInit(&argc, argv);           //inicializa o GLUT  
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB); //configura o modo de display (GLUT_DOUBLE dois buffer)  
    glutInitWindowPosition(200,0);  
    glutInitWindowSize(400, 400);    //configura a largura e altura da janela de exibição  
    glutCreateWindow("Move quadrado em x"); //cria a janela de exibição  
  
    glutSpecialFunc(special);         //chamada do teclado (teclas especiais, tipo seta)  
  
    init();                          //executa função de inicialização  
    glutDisplayFunc(display);        //estabelece a função "display" como a função callback de exibição  
    glutMainLoop();                 //mostre tudo e espere  
    return 0;  
}
```

Transformações 2D em OpenGL

- Exemplo 5 – translada (em x) apenas um dos dois objetos

```
void init(){  
    glClearColor(0.0, 0.0, 0.0, 0.0); //define a cor de fundo  
    glMatrixMode(GL_PROJECTION);  
  
    gluOrtho2D(0.0, 400.0, 0.0, 400.0); //define projeção ortogonal 2D que mapeia  
    // objetos da coordenada do mundo para coordenadas da tela  
}
```

Transformações 2D em OpenGL

- Exemplo 5 – translada (em x) apenas um dos dois objetos

```
//é um quadrado na origem e de tamanho unitário
void desenhaQuadrado() {
    glColor3f(1.0, 0.0, 0.0); //cor vermelho
    glBegin(GL_QUADS); //define o tipo de primitiva a ser desenhada
    //no centro do eixo de coordenadas (quadrado)
    glVertex2f(-0.5, -0.5); //coordenadas
    glVertex2f(-0.5, 0.5);
    glVertex2f(0.5, 0.5);
    glVertex2f(0.5, -0.5);
    glEnd();
}

void desenhaTriangulo() {
    glColor3f(0.0, 1.0, 0.0); //cor verde
    glBegin(GL_TRIANGLES); //define o tipo de primitiva a ser desenhada
    //no centro do eixo de coordenadas (triângulo)
    glVertex2f(-0.5, -0.5); //coordenadas
    glVertex2f(0.5, -0.5);
    glVertex2f(0.0, 0.5);
    glEnd(); //finaliza o desenho do triângulo
}
```

Transformações 2D em OpenGL

- **Exemplo 5 – translada (em x) apenas um dos dois objetos**
 - Aplicação do evento especial do teclado

```
GLfloat quadrado_x = 0;  
GLfloat quadrado_y = 0;
```

```
void special(int key, int x, int y){  
  
    switch(key){  
        case GLUT_KEY_LEFT:  
            quadrado_x -= 0.5;  
            break;  
        case GLUT_KEY_RIGHT:  
            quadrado_x += 0.5;  
            break;  
    }  
  
    glutPostRedisplay(); //redesenha a cena da função display  
}
```


Transformações 2D em OpenGL

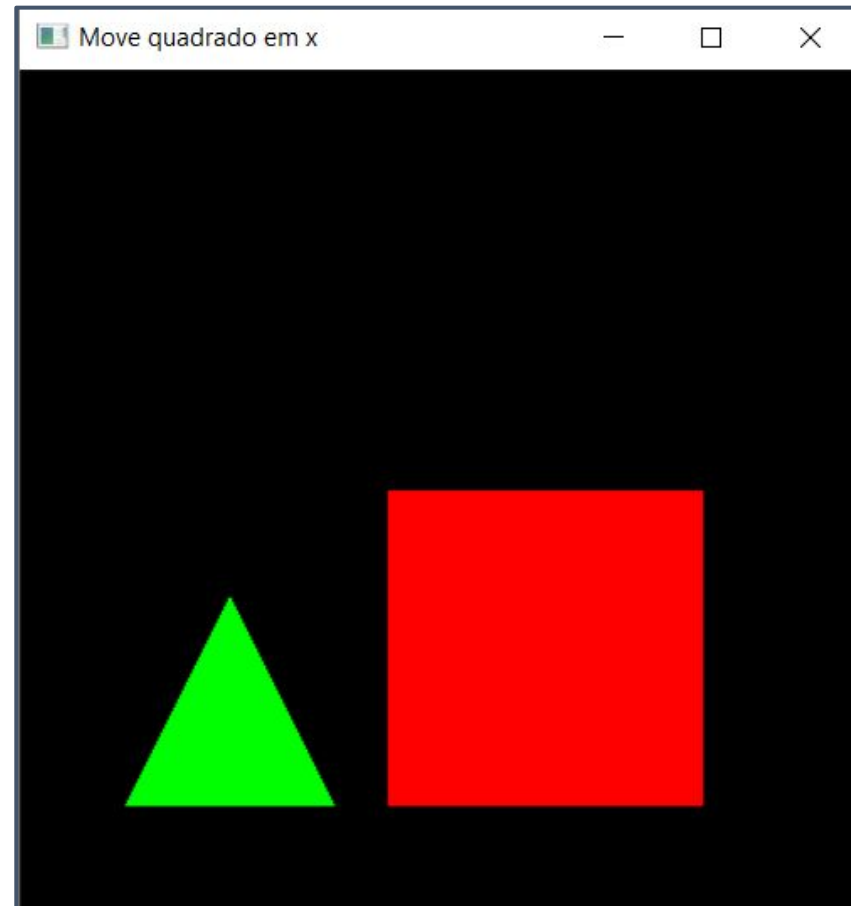
- Exemplo 5 – translada (em x) apenas um dos dois objetos

Deseja aplicar transformações locais a cada objeto individualmente, mas também deseja manter a transformação global da cena

```
void display() {  
  
    glClear(GL_COLOR_BUFFER_BIT);  
  
    glMatrixMode(GL_MODELVIEW);  
    glLoadIdentity(); //carrega a matriz identidade  
  
    glPushMatrix(); //cria matriz de transformações para o triângulo  
        glTranslatef(100, 100, 0);  
        glScalef(100, 100, 1);  
  
        desenhaTriangulo();  
    glPopMatrix();  
  
    glPushMatrix(); //cria matriz de transformações para o quadrado  
        glTranslatef(quadrado_x, 0, 0);  
        glTranslatef(250, 125, 0);  
        glScalef(150, 150, 1);  
  
        desenhaQuadrado();  
    glPopMatrix();  
  
    glutSwapBuffers(); //mudanca de buffer  
    //buffer duplo para evitar travar (intercala os buffers,  
    //um momento um só mostra e o outro só calcula)  
}
```


Transformações 2D em OpenGL

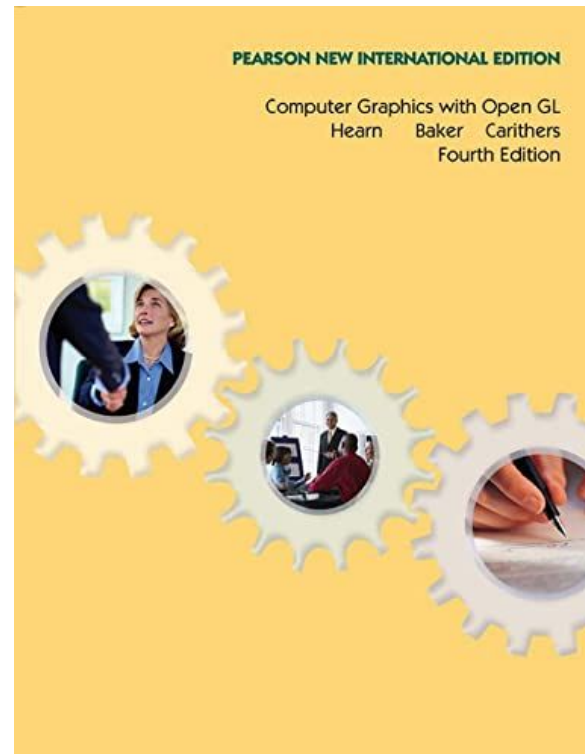
- **Exemplo 5 – translada (em x) apenas um dos dois objetos**



Referências



AZEVEDO, Eduardo; CONCI, Aura.
Computação gráfica: teoria e prática.
Elsevier, 2003.

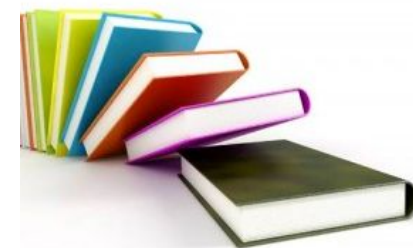


HEARN, DONALD. **Computer graphics with OpenGL.** Pearson Education Limited, 2014.
5º Edição.



PICHETTI, R. F; et al. **Computação Gráfica e Processamento de Imagens.**
Porto Alegre: SAGAH, 2022.

Referências



- ALENCAR, Aretha. **Notas de aula da disciplina Computação Gráfica da Universidade Tecnológica Federal do Paraná.** 2021.
- PAIVA, Anselmo. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2018.
- QUINTANILHA, Darlan. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2022.
- SILVA, Giovanni. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2019.